

Generazione di gloss ASL da testo inglese:
un'analisi neuro-simbolica
Fondamenti teorici, implementazione e risultati negativi con meccanismo

Progetto `neuro_symbolic_t2g`

8 settembre 2026

Sommario

Questa relazione documenta un sistema di traduzione automatica da inglese scritto a *gloss* della lingua dei segni americana (ASL), realizzato con un modello linguistico di piccole dimensioni (Qwen2.5-0.5B-Instruct) addestrato con *supervised fine-tuning* (SFT) e con *reinforcement learning* nella variante GRPO, sotto vincolo simbolico di decodifica.

Il risultato principale non è un miglioramento delle metriche, ma un **risultato negativo con meccanismo**: sul corpus impiegato (ASLG-PC12) le metriche di sovrapposizione lessicale sono *sature*, nel senso preciso che una baseline a regole di circa quindici righe di codice raggiunge ROUGE-L 0,9697 e supera ogni risultato ottenuto con reinforcement learning. Ne segue che la domanda di ricerca inizialmente posta — “il GRPO migliora la generazione di gloss?” — non è ben posta su questo corpus, e che i numeri pubblicati in letteratura su ASLG-PC12 vanno riletti alla luce di tale saturazione.

La relazione è scritta in modo autocontenuto: ogni nozione teorica di complessità media viene introdotta prima di essere usata, e ogni scelta implementativa è collegata al file e alla funzione che la realizzano.

Indice

1	Introduzione	5
1.1	Il problema: dal testo al gloss	5
1.2	Che cosa è stato costruito	5
1.3	Il risultato principale, in anticipo	6
1.4	Contributi	7
1.5	Come leggere questa relazione	7
2	Fondamenti teorici	8
2.1	Tokenizzazione: il testo come sequenza di simboli	8
2.2	Modelli linguistici autoregressivi	8
2.2.1	Architettura: Transformer, in breve	9
2.3	Addestramento supervisionato: entropia incrociata	9
2.3.1	Un obiettivo più debole: le etichette parziali	10
2.4	Adattamento efficiente: LoRA e quantizzazione	10
2.4.1	LoRA: adattamento a rango basso	10
2.4.2	QLoRA: pesi quantizzati	10
2.5	Generazione: strategie di decodifica	11
2.6	Strutture dati simboliche: il Trie	11
2.7	Apprendimento per rinforzo: impostazione	12
2.7.1	Il gradiente di policy	12
2.7.2	Baseline e vantaggio	12
2.7.3	Vincolo di prossimità e regolarizzazione KL	13
2.7.4	Perché la reward va progettata con sospetto	13
2.8	Nozioni di teoria dell'informazione	13
2.9	Metriche di valutazione: definizioni	13
2.10	Riepilogo della notazione	14
3	Il corpus: ASLG-PC12	15
3.1	Provenienza e struttura	15
3.1.1	Dimensioni e suddivisione	15
3.2	Il corpus è sintetico	16
3.2.1	Evidenza dalla letteratura	16
3.2.2	Evidenza interna: misure sul corpus	16
3.2.3	Quanta incertezza resta? Misura entropica	16
3.3	Artefatti del generatore	17
3.3.1	Perché il dato non è stato pulito	17
3.4	La baseline a regole	17
3.4.1	Costruzione	18
3.4.2	Risultati	18
3.4.3	Il controllo minimale: solo maiuscolo	18

3.5	Conseguenze per il disegno sperimentale	19
4	Modello e stack software	20
4.1	Il modello base	20
4.2	Adattamento: LoRA e quantizzazione	20
4.3	Lo stack software	21
4.4	Il formato del prompt	21
4.4.1	Zero-shot	21
4.4.2	Few-shot con retrieval	22
4.4.3	Perché questa distinzione è centrale	22
4.5	Il vocabolario di uscita	22
5	Supervised fine-tuning	23
5.1	L'obiettivo, in concreto	23
5.2	Configurazione	23
5.3	Risultati	24
5.4	Il confronto che conta: SFT contro regola	24
5.5	Che cosa impara l'SFT che la regola non sa	25
5.5.1	Il controllo: estendere la regola non funziona	25
5.6	L'SFT è al tetto del corpus	25
6	GRPO: apprendimento per rinforzo di gruppo	26
6.1	Dall'attore-critico al gruppo	26
6.1.1	La conseguenza immediata: i gruppi morti	26
6.2	Configurazione impiegata	27
6.2.1	Un dato di contesto: l'esposizione ai dati	27
6.3	I denominatori: quale obiettivo si sta ottimizzando	27
6.3.1	Che cosa è stato effettivamente eseguito	28
6.3.2	Un dettaglio semantico che si presta a fraintendimenti	28
6.3.3	Che cosa TRL 0.24.0 non implementa	29
6.4	La misura decisiva: il supporto dei rollout	29
6.4.1	La conclusione meccanica	30
6.5	Il curriculum e il suo generatore di numeri casuali	30
7	Il disegno delle reward	31
7.1	Il problema di progettazione	31
7.2	Lo stack di sette componenti	31
7.3	La telemetria: quali componenti portano informazione	32
7.4	Tre componenti rimosse, e perché	32
7.4.1	Argomento algebrico	32
7.4.2	Verifica empirica	32
7.5	Una componente nuova: <code>edit_validity</code>	33
7.5.1	Definizione	33
7.5.2	Test avversario 1: la reward premia la verbosità?	33
7.5.3	Test avversario 2: la calibrazione di w	33
7.5.4	Test avversario 3: quanto pesa il gate sul vocabolario	34
7.6	Riepilogo del metodo	34

8	Decodifica vincolata: il componente simbolico	36
8.1	Il problema: dal vincolo sulle parole al vincolo sui token	36
8.2	Implementazione	36
8.2.1	La doppia radice	36
8.2.2	Numeri della costruzione	37
8.2.3	L'interfaccia per gli usi non generativi	37
8.3	L'alternativa scartata: l'automa a stack	37
8.4	Il risultato controintuitivo	38
8.4.1	Come si spiega	38
8.4.2	L'effetto veramente importante: il vincolo crea il supporto	38
8.5	Il costo	39
8.6	Che cosa non fa il vincolo	39
8.7	Il costo nascosto della generazione vincolata	40
9	Obiettivi ausiliari	41
9.1	Obiettivo 1: la massa ammessa	41
9.1.1	Formulazione	41
9.1.2	Nota numerica	41
9.1.3	Che cosa questo obiettivo può e non può fare	42
9.2	Obiettivo 2: la loss strutturata	42
9.2.1	Formulazione	42
9.2.2	Vincolo di memoria	42
9.2.3	Qualificazione: quanto informa realmente il gloss precedente?	42
9.2.4	Il caveat, dichiarato	43
9.2.5	Onestà sulle ritrattazioni	43
9.3	Il contratto di non regressione	43
9.4	I criteri di promozione	44
10	Metriche di valutazione	45
10.1	Il difetto di ROUGE-L, misurato	45
10.1.1	Il maiuscolo non conta	45
10.1.2	I prefissi grammaticali vengono distrutti	46
10.1.3	La conclusione operativa	46
10.2	Le metriche primarie	46
10.2.1	Exact match normalizzato	46
10.2.2	Non-copy-token accuracy	46
10.3	Validity: la definizione, versione 2	47
10.4	Pass@ k	47
10.5	La metrica composta e la sua correzione	47
10.6	Incertezza: bootstrap clusterizzato	48
10.7	Il protocollo di valutazione, riassunto	48
11	Risultati	50
11.1	La tabella principale	50
11.2	Osservazione 1: nessun risultato con RL supera la regola	50
11.3	Osservazione 2: l'SFT è al tetto, il che invalida la metrica	51
11.4	Osservazione 3: il vincolo simbolico e le due metriche	51
11.5	Osservazione 4: le lunghezze non mostrano deriva verso la verbosità	51
11.6	Osservazione 5: la degradazione della pipeline SFT→GRPO	52
11.7	La decomposizione dell'effetto	52
11.8	Riepilogo per non-copy accuracy	53

12 Infrastruttura di calcolo	54
12.1 La catena di esecuzione	54
12.1.1 Il container: exec e non run	54
12.2 Il regime offline	55
12.3 La catena di job	55
12.4 Sei guasti reali	55
12.4.1 Guasto 1: percorso dei dati divergente fra test e produzione	55
12.4.2 Guasto 2: ambiente offline esportato dopo il primo Python	56
12.4.3 Guasto 3: cambio di firma in Transformers 5.3.0	56
12.4.4 Guasto 4: la coda invertita	56
12.4.5 Guasto 5: il generatore casuale globale nel curriculum	57
12.4.6 Guasto 6: celle di sola valutazione lanciate come addestramento	57
12.5 Il filo comune	57
13 Organizzazione del codice	59
13.1 Struttura dei moduli	59
13.2 Il sistema di configurazione	59
13.3 La matrice sperimentale	60
13.4 La validazione delle configurazioni	61
13.4.1 Le chiavi morte	61
13.4.2 Il vincolo sulla lunghezza del prompt	61
13.5 Le diagnostiche di Markov	61
13.6 Il servizio remoto	62
13.7 La suite di test	62
13.7.1 Un test rosso dichiarato	62
13.8 La pulizia del codice	63
14 Conclusioni	64
14.1 Che cosa è stato stabilito	64
14.1.1 Le metriche su ASLG-PC12 sono sature	64
14.1.2 La domanda di ricerca iniziale era mal posta	64
14.1.3 Il risultato negativo ha un meccanismo in tre parti	64
14.1.4 Il vincolo simbolico abilita l'apprendimento senza migliorare le metriche	65
14.1.5 L'onestà metodologica come parte del risultato	65
14.2 Lavoro futuro	66
14.2.1 Cambiare corpus	66
14.2.2 Ridisegnare l'esperimento primario	66
14.2.3 Eseguire i gate degli obiettivi ausiliari	67
14.2.4 Il reinforcement learning, se e quando	67
14.3 La lezione trasferibile	67

Capitolo 1

Introduzione

1.1 Il problema: dal testo al gloss

La lingua dei segni americana (*American Sign Language*, ASL) è una lingua naturale completa, con una propria grammatica, che non coincide con l'inglese. Non esiste una forma scritta ufficiale dell'ASL. Per lavorare con strumenti testuali si usa quindi una convenzione di trascrizione chiamata *gloss*: ogni segno viene rappresentato dalla parola inglese in maiuscolo che gli corrisponde più da vicino, eventualmente con marcatori aggiuntivi.

Un esempio reale tratto dal corpus impiegato in questo lavoro:

Inglese	they will not flinch .
Gloss	X-Y WILL DESC-NOT FLINCH .
Inglese	this kind of thing must never happen again .
Gloss	THIS KIND THING MUST DESC-NEVER HAPPEN DESC-AGAIN .

Va detto subito, perché condiziona tutta l'interpretazione dei risultati: il gloss **non** è la lingua dei segni. È una trascrizione lessicale impoverita, che perde la struttura spaziale, i marcatori non manuali (espressioni facciali, direzione dello sguardo) e la simultaneità tipica del segnato. Parte della letteratura sostiene che il gloss sia una rappresentazione inadeguata: Yin e Read [39] osservano che nei loro esperimenti la traduzione a partire dal *video* batte quella a partire dal gloss di riferimento, e concludono che il gloss è una rappresentazione inefficiente della lingua dei segni. Sul piano più generale del trattamento delle lingue dei segni nell'elaborazione del linguaggio naturale, Yin et al. [38] raccolgono le raccomandazioni metodologiche del settore, fra cui l'avvertenza a non trattare il gloss come se fosse la lingua.

Nonostante questi limiti il gloss resta l'interfaccia testuale su cui si costruiscono i sistemi esistenti: esistono baseline aperte basate su gloss per la traduzione da lingua parlata a lingua dei segni [16] e architetture di produzione che trattano il compito come selezione e riordino di segni [33].

Il compito affrontato qui, detto *text-to-gloss* (T2G), è dunque:

$$\text{inglese scritto} \mapsto \text{sequenza di gloss ASL} \quad (1.1)$$

Si tratta di un passo intermedio in una ipotetica pipeline più ampia (testo $\rightarrow$ gloss $\rightarrow$ avatar segnante), non di un sistema di traduzione completo.

1.2 Che cosa è stato costruito

Il sistema si basa su un modello linguistico di piccole dimensioni, *Qwen2.5-0.5B-Instruct*, adattato al compito con due famiglie di metodi:

- **SFT** (*supervised fine-tuning*): addestramento supervisionato classico, in cui il modello impara a riprodurre il gloss di riferimento;
- **GRPO** (*Group Relative Policy Optimization*): apprendimento per rinforzo, in cui il modello genera più candidati per ogni frase e viene premiato in base alla qualità relativa di ciascuno.

Le due famiglie sono state combinate in quattro celle sperimentali: modello base senza addestramento, solo SFT, solo GRPO a partire dal modello base, e la pipeline completa SFT seguito da GRPO.

A queste si sovrappongono due fattori ortogonali:

- la modalità di *prompting*: *zero-shot* (solo la frase da tradurre) oppure *few-shot* con retrieval (nel prompt vengono inseriti $k = 3$ esempi simili recuperati dal training set);
- la *decodifica vincolata*: un componente simbolico (un *Trie*, ossia un albero di prefissi) che a ogni passo di generazione vieta al modello di emettere token che non portino a un gloss valido.

Quest’ultimo elemento è ciò che rende il sistema *neuro-simbolico*: una rete neurale produce le probabilità, una struttura dati simbolica ne restringe il supporto.

1.3 Il risultato principale, in anticipo

Questa relazione non riporta un miglioramento delle metriche. Riporta un **risultato negativo con meccanismo**, che è più utile di un miglioramento apparente perché spiega *perché* la domanda iniziale era mal posta.

Il fatto centrale, misurato e documentato nel Capitolo 3, è che il corpus impiegato (ASLG-PC12) non è stato prodotto da annotatori umani: è **sintetico**, generato applicando regole all’inglese. La conseguenza pratica è che una baseline a regole, costruita in circa quindici righe di codice contando le corrispondenze parola-per-parola nel solo training set, ottiene:

Sistema	ROUGE-L	Exact match
Baseline a regole (nessun apprendimento)	0,9697	0,5912
SFT (modello addestrato)	0,9752	0,8130
GRPO da modello base	0,6076	0,0528

Si legga con attenzione la terza riga: il miglior risultato ottenuto con reinforcement learning (0,6076) sta *molto sotto* una regola che non apprende nulla (0,9697). Persino la trasformazione banale “metti tutto in maiuscolo” raggiunge ROUGE-L 0,6454, cioè batte il GRPO.

Da qui seguono tre conseguenze che strutturano il resto della relazione:

1. **Nessun numero è interpretabile senza la baseline a regole accanto.** Un risultato di 0,6 su questo corpus non è un successo parziale: è un fallimento rispetto a un riferimento che nessuno aveva calcolato.
2. **La domanda “il GRPO aiuta il T2G?” non è ben posta su ASLG-PC12.** Il compito è quasi deterministico, l’SFT lo satura, e qualsiasi metodo di rinforzo parte già in svantaggio rispetto a una regola. La domanda si riduce a “il GRPO memorizza una mappa quasi-identità meglio dell’SFT?”, che riguarda l’ottimizzazione e non la lingua dei segni.

3. **La metrica di riferimento è essa stessa difettosa.** Come mostrato nel Capitolo 10, l'implementazione di ROUGE-L impiegata è insensibile alle maiuscole e spezza i token sui caratteri non alfanumerici, quindi premia un modello che si limita a copiare l'inglese.

1.4 Contributi

Al netto del risultato negativo, il lavoro produce alcuni contributi verificabili:

- una **baseline a regole** riutilizzabile (`src/analysis/rule_baseline.py`) che rende interpretabili i numeri su questo corpus, insieme a una metrica insensibile alla copia (*non-copy-token accuracy*);
- un **censimento degli artefatti** del generatore del corpus: il 10,47% delle righe di training contiene parole corrotte da una rimozione di sottostringa (per esempio **therefore** diventa **DESC-REFORE**);
- la **quantificazione del supporto dei rollout**, che spiega meccanicamente quando il reinforcement learning non può funzionare (Capitolo 6);
- un risultato **neuro-simbolico non ovvio**: il vincolo simbolico peggiora la metrica di sovrapposizione e contemporaneamente *abilita* il segnale di reward (Capitolo 8);
- due **obiettivi ausiliari** implementati e qualificati prima di essere addestrati, con i relativi criteri di promozione (Capitolo 9).

1.5 Come leggere questa relazione

La relazione è pensata per essere autocontenuta. Il Capitolo 2 introduce da zero le nozioni teoriche necessarie: modelli linguistici autoregressivi, entropia incrociata, adattamento a rango basso, apprendimento per rinforzo, gradiente di policy. Chi ha già familiarità con questi argomenti può saltarlo.

I capitoli successivi seguono la pipeline: dati (Capitolo 3), modello (Capitolo 4), i due metodi di addestramento (Capitoli 5 e 6), il disegno delle reward (Capitolo 7), il componente simbolico (Capitolo 8), gli obiettivi ausiliari (Capitolo 9), le metriche (Capitolo 10) e i risultati (Capitolo 11). Gli ultimi tre capitoli riguardano l'infrastruttura di calcolo, l'organizzazione del codice e le conclusioni.

Ogni capitolo segue lo stesso schema: prima la teoria, poi la scelta implementativa, poi il file e la funzione che la realizzano. I valori numerici riportati sono tutti misurati; dove un valore non è disponibile è indicato esplicitamente con un segnaposto, invece di essere stimato.

Capitolo 2

Fondamenti teorici

Questo capitolo introduce da zero le nozioni necessarie a leggere il resto della relazione. Chi ha già familiarità con modelli linguistici autoregressivi, adattamento a rango basso e gradiente di policy può passare direttamente al Capitolo 3.

L'ordine è quello delle dipendenze logiche: prima come si rappresenta il testo (tokenizzazione), poi che cosa è un modello linguistico e come si addestra (entropia incrociata), poi come si adatta un modello grande con poche risorse (LoRA e quantizzazione), poi come si genera testo (decodifica), infine come si addestra con un segnale di qualità invece che con un riferimento (reinforcement learning e gradiente di policy).

2.1 Tokenizzazione: il testo come sequenza di simboli

Un modello linguistico non lavora su caratteri né su parole, ma su *token*: elementi di un vocabolario finito $\mathcal{V}$ costruito una volta per tutte a partire da un grande corpus. Un token può essere una parola intera, un frammento di parola, un segno di punteggiatura o uno spazio.

Formalmente, il tokenizzatore è una funzione

$$\text{tok} : \Sigma^* \longrightarrow \mathcal{V}^* \quad (2.1)$$

dove Σ è l'alfabeto dei caratteri e $\mathcal{V}^*$ l'insieme delle sequenze finite di token. Si richiede che tok sia invertibile (*lossless*): esiste detok tale che $\text{detok}(\text{tok}(s)) = s$ per ogni stringa s .

Questo dettaglio, apparentemente pedante, è centrale in questo progetto. La parola gloss `DESC-NOT` non è un token: viene spezzata in più token, per esempio `DESC`, `-`, `NOT`. Ne segue che un vincolo espresso a livello di *parola* ("emetti solo gloss appartenenti al vocabolario") deve essere tradotto in un vincolo a livello di *token* ("emetti solo token che siano prefisso valido di almeno un gloss"). È esattamente il problema che il Trie del Capitolo 8 risolve.

Un secondo dettaglio: molti tokenizzatori moderni codificano lo spazio come parte del token successivo. Il token che rappresenta "`HOUSE`" a inizio frase e quello che rappresenta " `HOUSE`" in mezzo a una frase sono identificatori *diversi*. Anche questo ha una conseguenza diretta (la doppia radice del Trie, sezione 8.2.1).

2.2 Modelli linguistici autoregressivi

Sia $y = (y_1, \dots, y_T)$ una sequenza di token. Un modello linguistico *autoregressivo* fattorizza la probabilità congiunta della sequenza usando la regola della catena:

$$p_\theta(y) = \prod_{t=1}^T p_\theta(y_t \mid y_{<t}), \quad y_{<t} = (y_1, \dots, y_{t-1}). \quad (2.2)$$

Nel caso condizionale che ci interessa (dato un ingresso x , l'inglese, produrre un'uscita y , il gloss) la fattorizzazione diventa

$$p_{\theta}(y \mid x) = \prod_{t=1}^T p_{\theta}(y_t \mid x, y_{<t}). \quad (2.3)$$

Il modello calcola, a ogni passo t , un vettore di *logits* $z_t \in \mathbb{R}^{|\mathcal{V}|}$, cioè un punteggio non normalizzato per ogni token del vocabolario. La distribuzione di probabilità si ottiene applicando la funzione softmax:

$$p_{\theta}(y_t = v \mid x, y_{<t}) = \frac{\exp(z_{t,v})}{\sum_{u \in \mathcal{V}} \exp(z_{t,u})}. \quad (2.4)$$

Il denominatore, detto *funzione di partizione*, garantisce che le probabilità sommino a uno. Il fatto che la somma sia su *tutto* il vocabolario è ciò che rende non banale imporre un vincolo: se si vietano alcuni token, il denominatore va ricalcolato sull'insieme ammesso.

2.2.1 Architettura: Transformer, in breve

L'architettura impiegata (Qwen2.5, si veda il Capitolo 4) è un Transformer *decoder-only*. Non è necessario, per leggere questa relazione, conoscerne i dettagli; bastano tre fatti.

1. Ogni token viene trasformato in un vettore (*embedding*) e passa attraverso una pila di blocchi identici nella struttura.
2. Ogni blocco contiene un modulo di *attenzione*, che mette in relazione la posizione corrente con tutte le posizioni precedenti, e un modulo *feed-forward* (una rete densa a due strati con non linearità). I moduli dell'attenzione sono chiamati per convenzione `q_proj`, `k_proj`, `v_proj`, `o_proj`; quelli feed-forward `gate_proj`, `up_proj`, `down_proj`. Sono esattamente i sette moduli su cui agisce l'adattamento LoRA descritto più avanti.
3. L'attenzione è *causale*: la posizione t non può guardare le posizioni $> t$. È questa mascheratura che rende valida la fattorizzazione (2.3) e che permette di calcolare, in un unico passaggio, le probabilità di tutti i token di una sequenza nota.

2.3 Addestramento supervisionato: entropia incrociata

Dato un insieme di esempi $\mathcal{D} = \{(x^{(i)}, y^{(i)})\}_{i=1}^N$, l'obiettivo dell'addestramento supervisionato è massimizzare la log-verosimiglianza dei riferimenti, ossia minimizzare la *loss* di entropia incrociata:

$$\mathcal{L}_{\text{CE}}(\theta) = -\frac{1}{N} \sum_{i=1}^N \sum_{t=1}^{T_i} \log p_{\theta}(y_t^{(i)} \mid x^{(i)}, y_{<t}^{(i)}). \quad (2.5)$$

Interpretazione: per ogni posizione si guarda quale probabilità il modello assegna al token *corretto* e si penalizza il logaritmo negativo di quella probabilità. Se il modello assegna probabilità 1 al token corretto il contributo è 0; se assegna probabilità prossima a 0 il contributo diverge.

Due proprietà rilevanti nel seguito:

Teacher forcing. Nella (2.5) il condizionamento è sul *prefisso di riferimento* $y_{<t}^{(i)}$, non su quanto il modello avrebbe generato. Questo rende l'addestramento parallelizzabile su tutte le posizioni, ma crea una discrepanza fra addestramento e generazione (*exposure bias*): in generazione il modello condiziona sui propri errori.

Mascheratura del prompt. Nel nostro caso interessa solo che il modello impari a produrre il gloss, non a riprodurre l'inglese di ingresso. Le posizioni corrispondenti al prompt vengono quindi *escluse* dalla somma. Nella pratica implementativa si assegna a quelle posizioni l'etichetta convenzionale -100 , che le librerie usate interpretano come “ignora” (si veda il Capitolo 5).

2.3.1 Un obiettivo più debole: le etichette parziali

Esiste una variante utile quando non si conosce il token corretto, ma si conosce un *insieme* $\mathcal{A}_t \subseteq \mathcal{V}$ che lo contiene. In quel caso si può massimizzare la probabilità che il modello assegni all'insieme, invece che a un singolo elemento:

$$\mathcal{L}_{\text{AM}}(\theta) = -\log \sum_{v \in \mathcal{A}_t} p_\theta(v \mid x, y_{<t}). \quad (2.6)$$

Questa è la formulazione classica dell'apprendimento a etichette parziali (*partial-label learning*, [2]) e coincide con la massimizzazione della verosimiglianza marginale (*maximum marginal likelihood*, [8]). Va sottolineata una proprietà che sarà decisiva nel Capitolo 9: la (2.6) è *piatta* sull'insieme ammesso. Qualunque ridistribuzione della massa *interna* ad $\mathcal{A}_t$ lascia la loss invariata. Ne segue che questo obiettivo può migliorare la *calibrazione* (quanta probabilità va sui token legali) ma non può, da solo, migliorare l'exact match.

2.4 Adattamento efficiente: LoRA e quantizzazione

Aggiornare tutti i parametri di un modello linguistico richiede memoria per i pesi, per i gradienti e per gli stati dell'ottimizzatore. Due tecniche riducono drasticamente questo costo.

2.4.1 LoRA: adattamento a rango basso

L'idea di LoRA (*Low-Rank Adaptation*, [11]) è che l'aggiornamento necessario per adattare un modello pre-addestrato a un compito specifico abbia *rango intrinseco basso*. Invece di aggiornare una matrice di pesi $W_0 \in \mathbb{R}^{d \times k}$, si congela W_0 e si apprende un aggiornamento fattorizzato:

$$W = W_0 + \Delta W, \quad \Delta W = \frac{\alpha}{r} BA, \quad (2.7)$$

con $A \in \mathbb{R}^{r \times k}$, $B \in \mathbb{R}^{d \times r}$ e $r \ll \min(d, k)$. Il numero di parametri addestrabili passa da dk a $r(d+k)$.

I due iperparametri hanno un significato preciso:

- r (*rango*) fissa la capacità dell'adattamento;
- α (*alpha*) è un fattore di scala; il rapporto α/r nella (2.7) serve a rendere l'ampiezza dell'aggiornamento indipendente dalla scelta di r , così che cambiare r non richieda di ritardare il learning rate.

All'inizializzazione $B = 0$, quindi $\Delta W = 0$ e il modello adattato coincide esattamente con quello di partenza.

2.4.2 QLoRA: pesi quantizzati

La quantizzazione riduce la precisione numerica con cui i pesi congelati sono memorizzati. In QLoRA [3] i pesi base W_0 sono memorizzati a 4 bit, mentre gli adattatori A, B restano in

precisione più alta e sono gli unici a ricevere gradiente. Il risultato è che un modello che in precisione a 16 bit non entrerebbe in memoria diventa addestrabile su una sola GPU.

Il costo è un rumore di quantizzazione sui pesi base. Per il presente lavoro la conseguenza operativa è che i risultati non sono bit-per-bit riproducibili cambiando backend di quantizzazione, mentre lo sono a parità di backend e seed.

2.5 Generazione: strategie di decodifica

Data la distribuzione (2.4), produrre una sequenza richiede una regola di scelta. Le opzioni rilevanti qui sono tre.

Greedy. Si prende a ogni passo il token più probabile, $y_t = \arg \max_v p_\theta(v \mid x, y_{<t})$. È deterministico ma tende a produrre testo ripetitivo [10, 34].

Campionamento con temperatura. Si campiona da una distribuzione riscalata:

$$p_\theta^{(\tau)}(v) \propto \exp\left(\frac{z_{t,v}}{\tau}\right). \quad (2.8)$$

Per $\tau \rightarrow 0$ si recupera il greedy; per $\tau = 1$ la distribuzione del modello; per $\tau > 1$ una distribuzione più piatta, quindi più varia. La temperatura è il principale strumento per controllare la diversità dei candidati, e nel reinforcement learning è ciò che determina se il gruppo di candidati contiene o no differenze su cui apprendere.

Decodifica vincolata. Si azzerla la probabilità dei token non ammessi prima della normalizzazione. Operativamente si pone $z_{t,v} \leftarrow -\infty$ per $v \notin \mathcal{A}_t$, cosicché

$$p_\theta(v \mid x, y_{<t}, \mathcal{A}_t) = \begin{cases} \frac{\exp(z_{t,v})}{\sum_{u \in \mathcal{A}_t} \exp(z_{t,u})} & v \in \mathcal{A}_t, \\ 0 & \text{altrimenti.} \end{cases} \quad (2.9)$$

Questa è la forma di generazione con *lessico chiuso* adottata nel Capitolo 8, ed è imparentata con la decodifica grammaticalmente vincolata [5, 23] e con la decodifica a vincoli lessicali [9, 26].

Va osservato che la (2.9) *non* è una semplice riparametrizzazione: sposta massa di probabilità dai token vietati a quelli ammessi in proporzione ai logits originali. Il modello non “sa” di essere vincolato, dunque il vincolo può produrre sequenze che il modello non avrebbe mai generato liberamente. È il motivo per cui il vincolo può *peggiorare* una metrica di sovrapposizione e contemporaneamente *migliorare* la validità strutturale.

2.6 Strutture dati simboliche: il Trie

Un *Trie* (o albero dei prefissi) è un albero in cui ogni nodo rappresenta un prefisso e ogni arco è etichettato con un simbolo. Dato un vocabolario finito di stringhe G (nel nostro caso: l’insieme dei gloss ammessi), il Trie costruito su G permette di rispondere in tempo proporzionale alla lunghezza del prefisso alla domanda:

“dato il prefisso di token s , quali token possono seguire senza rendere impossibile il completamento in un elemento di G ?”

Formalmente, detto $\text{Pref}(G)$ l'insieme dei prefissi di token degli elementi di G , l'insieme ammesso è

$$\mathcal{A}(s) = \{ v \in \mathcal{V} : s \cdot v \in \text{Pref}(G) \} \quad (2.10)$$

dove $\cdot$ denota la concatenazione. Il Trie è precisamente la struttura che materializza $\text{Pref}(G)$ in modo che la (2.10) sia calcolabile con un accesso a un dizionario.

Un Trie riconosce il linguaggio G , che è finito e quindi regolare. Un automa a stack (*pushdown automaton*) riconosce linguaggi context-free, strettamente più espressivi. La scelta fra i due, nel Capitolo 8, è motivata dal fatto che il linguaggio target di questo progetto è **gloss***, cioè una semplice chiusura di Kleene su un insieme finito: un Trie è sufficiente, e un automa a stack sarebbe sovradimensionato.

2.7 Apprendimento per rinforzo: impostazione

Nell'addestramento supervisionato si dispone del riferimento $y^{(i)}$. Nel reinforcement learning si dispone invece di un *segnale di qualità*, la *reward*, che valuta un'uscita prodotta dal modello stesso.

Si modella la generazione come un processo decisionale:

- lo *stato* al passo t è il prefisso $(x, y_{<t})$;
- l'*azione* è la scelta del token y_t ;
- la *policy* è il modello stesso, $\pi_\theta(y_t \mid x, y_{<t})$;
- la *reward* $R(x, y) \in \mathbb{R}$ è assegnata alla sequenza completa (*reward terminale*).

L'obiettivo è massimizzare la reward attesa:

$$J(\theta) = \mathbb{E}_{x \sim \mathcal{D}} \mathbb{E}_{y \sim \pi_\theta(\cdot \mid x)} [R(x, y)]. \quad (2.11)$$

2.7.1 Il gradiente di policy

Il problema tecnico è che la variabile rispetto a cui si deriva, θ , compare nella distribuzione su cui si prende l'aspettazione. La soluzione classica è l'identità del *likelihood ratio*: poiché $\nabla_\theta \pi_\theta(y) = \pi_\theta(y) \nabla_\theta \log \pi_\theta(y)$, si ottiene

$$\nabla_\theta J(\theta) = \mathbb{E}_{y \sim \pi_\theta} [R(x, y) \nabla_\theta \log \pi_\theta(y \mid x)]. \quad (2.12)$$

Questo è il teorema del gradiente di policy, alla base di REINFORCE [35]. Il significato è diretto: si aumenta la log-probabilità delle sequenze con reward alta e si diminuisce quella delle sequenze con reward bassa.

2.7.2 Baseline e vantaggio

La stima (2.12) è corretta ma ha varianza elevata. Si dimostra che sottrarre una qualunque quantità b indipendente dall'azione lascia il gradiente inalterato in aspettazione, poiché $\mathbb{E}_{y \sim \pi_\theta} [\nabla_\theta \log \pi_\theta(y)] = 0$. Si definisce allora il *vantaggio*

$$A(x, y) = R(x, y) - b(x) \quad (2.13)$$

e si sostituisce R con A nella (2.12). La scelta di $b(x)$ è ciò che distingue gli algoritmi fra loro: PPO [29], impiegato anche nella messa a punto di modelli linguistici con feedback umano [21], apprende una rete separata (il *critic*) che stima $b(x)$; GRPO [31] usa invece la media delle

reward di un gruppo di candidati generati per lo stesso x , eliminando il critic. Il dettaglio è nel Capitolo 6.

Da (2.13) segue un fatto che ricorrerà come chiave interpretativa di tutta la parte sperimentale: se *tutti* i candidati per un dato x ricevono la stessa reward, allora $A = 0$ per ciascuno e il gradiente è nullo. Quel prompt non contribuisce all'apprendimento. Chiameremo *gruppo morto* un gruppo in questa condizione, e *supporto* la frazione di gruppi che producono un gradiente non nullo.

2.7.3 Vincolo di prossimità e regolarizzazione KL

Aggiornamenti troppo grandi degradano la policy in modo irreversibile. Si introduce quindi un termine che penalizza l'allontanamento da una policy di riferimento π_{ref} (tipicamente il modello prima dell'addestramento per rinforzo):

$$J_{\beta}(\theta) = \mathbb{E}[A(x, y)] - \beta D_{\text{KL}}(\pi_{\theta} \parallel \pi_{\text{ref}}), \quad (2.14)$$

dove $D_{\text{KL}}(p \parallel q) = \sum_v p(v) \log \frac{p(v)}{q(v)} \geq 0$ è la divergenza di Kullback-Leibler, nulla se e solo se $p = q$. Il coefficiente β regola quanto la policy è libera di allontanarsi dal riferimento.

2.7.4 Perché la reward va progettata con sospetto

Ottimizzare una reward imperfetta produce comportamenti che massimizzano la misura senza migliorare l'obiettivo reale. È il fenomeno noto come *reward hacking* o *specification gaming* [1, 4]; un caso documentato e pertinente è la deriva verso risposte più lunghe quando la reward correla con la lunghezza [32]. La teoria classica del *reward shaping* [18] caratterizza quali trasformazioni della reward preservano la politica ottima; nella pratica di questo progetto la contromisura adottata è diversa e più diretta: ogni componente di reward viene sottoposta a test avversari costruiti a mano (Capitolo 7).

2.8 Nozioni di teoria dell'informazione

Due quantità servono nel seguito per misurare quanta incertezza esista in un compito.

L'*entropia* di una variabile discreta Y misura l'incertezza media, in bit:

$$H(Y) = - \sum_y p(y) \log_2 p(y). \quad (2.15)$$

L'*entropia condizionale* misura l'incertezza residua su Y una volta noto X :

$$H(Y \mid X) = - \sum_{x,y} p(x, y) \log_2 p(y \mid x). \quad (2.16)$$

La differenza $H(Y) - H(Y \mid X) = I(X; Y)$ è l'informazione mutua: quanta incertezza su Y viene rimossa dall'osservazione di X . Nel Capitolo 3 queste quantità vengono stimate empiricamente sul corpus per rispondere a una domanda concreta: quanta incertezza resta sul gloss una volta noto il token inglese corrispondente? La risposta (0,856 bit su 8,578) quantifica in che senso il compito sia quasi deterministico.

2.9 Metriche di valutazione: definizioni

Le metriche usate nel seguito vanno definite qui, perché la loro forma esatta determina l'interpretazione dei risultati.

Exact match (EM). Frazione di uscite identiche al riferimento dopo normalizzazione. È una metrica binaria e severa, ma non manipolabile.

ROUGE-L. Basata sulla più lunga sottosequenza comune (*longest common subsequence*, LCS) fra generato g e riferimento r [14]. Con $P = \text{LCS}(g, r)/|g|$ e $R = \text{LCS}(g, r)/|r|$, la misura F è

$$F_1 = \frac{2PR}{P + R}. \quad (2.17)$$

Essendo una sottosequenza (non una sottostringa), l'ordine conta ma non la contiguità.

BLEU. Media geometrica delle precisioni su n -grammi, con una penalità per uscite troppo brevi [22]. Va distinta la variante *a frase* (mediata sulle frasi) da quella *a corpus* (con conteggi aggregati): danno numeri diversi e vanno riportate separatamente.

chrF. Analoga a BLEU ma su n -grammi di *caratteri* [25]; più robusta alla morfologia.

Pass@ k . Probabilità che almeno una fra k generazioni indipendenti superi una soglia di correttezza. Dato che su n campioni si osservano c successi, lo stimatore non distorto è

$$\text{pass}@k = 1 - \frac{\binom{n-c}{k}}{\binom{n}{k}}. \quad (2.18)$$

Un avvertimento che il Capitolo 10 svilupperà in dettaglio: tutte le metriche di questa lista, tranne l'exact match, sono metriche di *sovrapposizione*. Su un compito in cui l'uscita corretta è in larga parte una copia dell'ingresso, esse assegnano punteggi alti a sistemi che non hanno imparato nulla. Per questo il progetto introduce una metrica complementare, la *non-copy-token accuracy*, che valuta solo le posizioni in cui il gloss *differisce* dal testo di partenza.

2.10 Riepilogo della notazione

Simbolo	Significato
x	frase inglese di ingresso (prompt)
y	sequenza di gloss generata
$\mathcal{V}$	vocabolario di token del modello
$\mathcal{A}_t$	insieme dei token ammessi al passo t
π_θ	policy corrente (il modello addestrato)
π_{ref}	policy di riferimento (modello prima del RL)
z_t	logits al passo t
$R(x, y)$	reward della sequenza completa
$A(x, y)$	vantaggio
G	numero di candidati per gruppo (GRPO)
r, α	rango e scala di LoRA
β	coefficiente della penalità KL
τ	temperatura di campionamento

Capitolo 3

Il corpus: ASLG-PC12

Questo capitolo è il più importante della relazione. Non perché descriva l'ingegneria più complessa, ma perché ogni conclusione dei capitoli successivi dipende da un fatto stabilito qui: il corpus impiegato non è stato prodotto da annotatori umani, ed è quasi deterministico. Tutte le metriche di sovrapposizione lessicale che si possono calcolare su di esso sono, in senso tecnico, sature.

3.1 Provenienza e struttura

Il corpus è ASLG-PC12 [19], ottenuto allineando il corpus parlamentare inglese con una versione in gloss ASL. Ogni riga è una coppia:

Campo	Esempio
text	they will not flinch .
gloss	X-Y WILL DESC-NOT FLINCH .

I marcatori con prefisso hanno una funzione grammaticale: X- indica un pronome (X-Y per *they*, X-WE per *we*, X-I per *I*), mentre DESC- indica un descrittore, tipicamente un avverbio o una negazione (DESC-NOT, DESC-NEVER, DESC-AGAIN).

3.1.1 Dimensioni e suddivisione

Grandezza	Valore
Righe totali (dato grezzo)	87 710
Suddivisione	90/10, seed 42
Righe di training	72 979
Righe di test	8 109
Vocabolario gloss (token distinti)	16 121
Vocabolario gloss dopo filtraggio	15 472

Le due cifre del vocabolario differiscono perché il filtro scarta forme che non possono essere emesse (si veda il Capitolo 8); l'insieme effettivamente usato dal vincolo di decodifica conta 15 472 gloss.

Il seed 42 e la proporzione 90/10 sono fissati nel codice, non scelti per esperimento: ogni numero riportato in questa relazione si riferisce a questa suddivisione. Dove è stato necessario rielezionare soglie, è stato usato un *secondo* livello di suddivisione interno al solo training set (90/10, seed 1234), per non contaminare il test.

3.2 Il corpus è sintetico

Questo è il punto centrale. ASLG-PC12 non è stato annotato da segnanti: è stato generato applicando regole all'inglese. La conseguenza è che la relazione fra **text** e **gloss** è in larga parte una trasformazione meccanica invertibile, non una traduzione fra due lingue.

L'evidenza è di due tipi: esterna (letteratura) e interna (misure sul corpus).

3.2.1 Evidenza dalla letteratura

Moryossef et al. [17] documentano la natura sintetica del corpus e il suo uso come dato aumentato piuttosto che come riferimento.

Più diretta è l'analisi di Peng et al. [24]. Nella loro Tabella 7 riportano la sovrapposizione lessicale fra i token del gloss e quelli del testo:

Corpus	Sovrapposizione token gloss/testo
ASLG-PC12	98,23%
PHOENIX-2014T	15,59%

Un fattore sei di differenza. Su ASLG-PC12 il gloss usa quasi esattamente le stesse parole del testo; su un corpus annotato da umani (PHOENIX-2014T, lingua dei segni tedesca) no.

Nella loro Tabella 3, gli stessi autori mostrano che una *regola scritta a mano* ottiene BLEU 96,75 su ASLG-PC12. Un valore di questo ordine su un compito di traduzione automatica non significa che il sistema traduca bene: significa che non c'è traduzione da fare.

3.2.2 Evidenza interna: misure sul corpus

Sono state effettuate misure indipendenti sul dato in nostro possesso, per quantificare esattamente quanto il gloss sia una copia del testo. I risultati:

Misura	Valore
Token gloss identici al token sorgente maiuscolizzato	61,79%
idem, ignorando i prefissi DESC- e X-	79,74%
idem, confrontando i primi 4 caratteri (<i>stem</i>)	90,26%
Righe con <code>len(text) == len(gloss)</code> (in token)	27,5%

La lettura della prima colonna è progressiva. Quasi due token su tre sono letteralmente la parola inglese in maiuscolo. Se si perdona il prefisso grammaticale si arriva a quattro su cinque. Se si perdona la desinenza (usando solo la radice di quattro caratteri) si arriva a nove su dieci.

Solo il 27,5% delle righe ha la stessa lunghezza in token nelle due colonne. Questo dato ha un'importanza tecnica specifica: significa che l'allineamento posizionale uno-a-uno fra testo e gloss è disponibile solo su un quarto del corpus. Tutte le analisi che richiedono un allineamento (per esempio la stima delle transizioni nel Capitolo 9) sono limitate a quel sottoinsieme, e questo va tenuto presente come *caveat* nella lettura dei risultati.

3.2.3 Quanta incertezza resta? Misura entropica

La domanda si può porre in modo preciso con gli strumenti della sezione sui fondamenti di teoria dell'informazione. Sia Y il token gloss e X il token sorgente allineato. Le stime empiriche sono:

Quantità	Valore
$H(Y)$, entropia del token gloss	8,578 bit
$H(Y X)$, entropia condizionata al token sorgente	0,856 bit
$I(X; Y)$, informazione mutua	7,722 bit

Cioè: conoscere il token inglese rimuove circa il 90% dell'incertezza sul token gloss. Resta meno di un bit di ambiguità residua per token.

Questa singola riga spiega, da sola, la maggior parte dei risultati sperimentali:

- una tabella di corrispondenze uno-a-uno cattura quasi tutto il segnale (sezione 3.4);
- l'SFT satura il compito rapidamente (Capitolo 5);
- il reinforcement learning non ha spazio per migliorare, perché non c'è una scelta difficile da fare (Capitolo 6).

3.3 Artefatti del generatore

Poiché il corpus è generato da regole, ne eredita i difetti. Il più vistoso è una rimozione di sottostringa applicata senza vincoli di confine di parola: la sequenza di caratteri **the** viene rimossa *anche quando è interna a un'altra parola*.

Parola inglese	Gloss prodotto	Occorrenze (training)
therefore	DESC-REFORE	1 875
these	SE	2 847
there	DESC-RE	3 279

In totale 7641 righe di training, ossia il 10,47%, contengono almeno una parola corrotta da questo meccanismo. Una riga su dieci del riferimento contiene una forma che nessun segnante userebbe.

Un secondo artefatto è la presenza del token **20the**, residuo evidente di una decodifica mancata della sequenza percent-encoded **%20the** nell'URL o nel file di origine.

Un terzo riguarda la codifica dei caratteri: 147 righe su 87710 (0,168%) contengono caratteri non ASCII, e 72 token del vocabolario gloss sono non ASCII (fra cui una forma che inizia con il carattere di *byte order mark*).

3.3.1 Perché il dato non è stato pulito

È una scelta deliberata, e va motivata perché a prima vista appare controintuitiva.

Ripulire questi artefatti cambierebbe il compito. I numeri di riferimento pubblicati in letteratura su ASLG-PC12 sono calcolati sul dato *così come è*; ripulirlo renderebbe i nostri risultati non confrontabili con quelli. Poiché il contributo principale di questo lavoro è mostrare che quei numeri pubblicati sono saturi, è necessario misurare sullo stesso dato che li ha prodotti.

Gli artefatti sono quindi documentati e conservati. Costituiscono anzi un argomento a favore della tesi centrale: un corpus con il 10% di righe corrotte da un bug di sostituzione di stringhe non è un riferimento adeguato per valutare la qualità di una traduzione.

3.4 La baseline a regole

Se il corpus è quasi deterministico, la domanda naturale è: quanto arriva lontano una regola? La risposta è il riferimento più importante di tutta la relazione.

3.4.1 Costruzione

L'implementazione è in `src/analysis/rule_baseline.py`. Il metodo è elementare:

1. si scorre il *solo training set* e, sulle righe in cui testo e gloss hanno la stessa lunghezza in token, si contano le corrispondenze posizionali parola $\rightarrow$ gloss;
2. per ogni parola inglese si conserva il gloss più frequente, ottenendo un lessico uno-a-uno di 10 996 voci;
3. si apprendono, con lo stesso conteggio, 13 *cancellazioni*: parole inglesi che nel gloss non compaiono (fra cui **the**, **of** e il già citato **20the**);
4. in inferenza si applica il lessico parola per parola, senza guardare il contesto.

Si noti che la baseline è *context-free*: traduce ogni parola indipendentemente dalle altre. Non c'è modello, non c'è addestramento, non ci sono parametri oltre alla tabella.

3.4.2 Risultati

Sul test set completo (8 109 righe):

Metrica	Valore
ROUGE-L	0,9697
Exact match	0,5912
Non-copy-token accuracy	0,9428

Sul sottoinsieme di 2 000 prompt usato per la valutazione dei modelli (per consentire il confronto diretto con le tabelle del Capitolo 11):

Metrica	Valore
ROUGE-L	0,9685
Exact match	0,5865
Gloss F1	0,9665
BLEU (a frase)	0,8846
BLEU (a corpus)	0,8907
chrF	96,47

Le soglie della baseline sono state rielezionate sulla suddivisione interna di sviluppo (90/10, seed 1234, ricavata dal solo training set) per escludere che il risultato dipenda da una scelta fortunata: il risultato è rimasto invariato.

3.4.3 Il controllo minimale: solo maiuscolo

Per isolare quanto del punteggio derivi dal semplice fatto che il gloss è maiuscolo, è stata valutata la trasformazione `uppercase(text)` senza alcun lessico:

Sistema	ROUGE-L
<code>uppercase(text)</code>	0,6454
Baseline a regole	0,9685

Il risultato è duplice. Da un lato, il lessico aggiunge un contributo reale ($0,65 \rightarrow 0,97$): la baseline non è banale. Dall'altro, e questo è il punto scomodo, un ROUGE-L di 0,6454 è ottenibile *senza fare nulla*, e come si vedrà nel Capitolo 11 supera il miglior risultato ottenuto con reinforcement learning (0,6076).

3.5 Conseguenze per il disegno sperimentale

Da questo capitolo derivano quattro vincoli metodologici, che sono stati applicati a tutte le misure riportate nel seguito.

1. **Nessuna metrica va riportata senza la baseline a regole accanto.** La baseline è il livello zero: un sistema che non la supera non ha imparato nulla di utile su questo corpus.
2. **Servono metriche insensibili alla copia.** La non-copy-token accuracy, definita nel Capitolo 10, valuta solo le posizioni in cui il gloss differisce dal testo. È l'unica metrica di sovrapposizione che discrimina fra i sistemi in modo interpretabile.
3. **Il confronto con i numeri pubblicati va contestualizzato.** Il miglior ROUGE-L pubblicato su ASLG-PC12 è 95,87 [24], cioè *sotto* la nostra baseline a regole. Non è un errore degli autori: è una proprietà del corpus.
4. **Le conclusioni non si trasferiscono ad altri corpora.** Su PHOENIX-2014T, dove la sovrapposizione è 15,59%, i migliori risultati T2G riportati sono intorno a ROUGE-L 55,18 [20]. Quello è un compito reale; questo no.

Capitolo 4

Modello e stack software

4.1 Il modello base

Il modello impiegato è **Qwen2.5-0.5B-Instruct**: un Transformer decoder-only da circa 0,5 miliardi di parametri, nella variante già sottoposta a istruzione (*instruct*), cioè addestrata a seguire richieste formulate in linguaggio naturale.

La scelta di un modello piccolo è motivata da tre ragioni, in ordine di peso:

1. **Vincolo di risorse.** L'addestramento gira su una singola GPU di un cluster condiviso con una quota di una sola allocazione per volta (Capitolo 12). Il reinforcement learning richiede di generare G candidati per ogni prompt a ogni passo, quindi il costo di generazione domina; un modello grande renderebbe l'esperimento infattibile.
2. **Adeguatezza al compito.** Come stabilito nel Capitolo 3, il compito ha meno di un bit di incertezza residua per token. Non è un compito che richiede capacità di ragionamento: richiede di memorizzare una mappa lessicale.
3. **Iterazione.** Un modello piccolo permette cicli di esperimento brevi, che è ciò che serve quando l'oggetto dell'indagine è il *disegno* sperimentale e non la prestazione assoluta.

La variante *instruct* è preferita alla variante base perché il prompt adottato ha forma di istruzione, e perché nella cella *few-shot* gli esempi sono inseriti come turni di conversazione.

4.2 Adattamento: LoRA e quantizzazione

Il modello base non viene aggiornato. Si applica l'adattamento a rango basso descritto nella (2.7) con la configurazione seguente.

Iperparametro	Valore
Rango r	32
Scala α	64
Dropout	0,05
Moduli adattati	7
Quantizzazione dei pesi base	4 bit (QLoRA)

I sette moduli sono **q_proj**, **k_proj**, **v_proj**, **o_proj** (le quattro proiezioni dell'attenzione) e **gate_proj**, **up_proj**, **down_proj** (le tre proiezioni del blocco feed-forward). Adattare anche il blocco feed-forward, e non solo l'attenzione, è la configurazione che la letteratura

su QLoRA [3] indica come preferibile quando il numero di parametri addestrabili non è il vincolo stringente.

Il rapporto $\alpha/r = 2$ è la scelta convenzionale. Il dropout 0,05 agisce solo sul ramo dell'adattatore.

Il caricamento è realizzato in `src/models/model_loader.py`, che incapsula sia la quantizzazione a 4 bit sia l'installazione degli adattatori, esponendo una sola funzione di caricamento usata da tutti i punti di ingresso (SFT, GRPO, valutazione). Questa centralizzazione è deliberata: garantisce che la cella di valutazione carichi il modello esattamente come la cella di addestramento, evitando una classe di discrepanze difficile da diagnosticare.

4.3 Lo stack software

La riproducibilità richiede di fissare le versioni. Lo stack è il seguente.

Componente	Versione	Ruolo
PyTorch	2.7.1+cu118	tensori, autograd
CUDA	11,8	runtime GPU
Transformers	5,3,0	modello, tokenizzatore, generazione
TRL	0,24,0	SFTTrainer , GRPOTrainer
PEFT	0,19,1	implementazione LoRA
Unsloth	—	kernel ottimizzati, caricamento 4 bit
torchao	0,17,0	primitive di quantizzazione
sacrebleu	2,6,0	BLEU e chrF
sentence-transformers	5,2,3	retrieval per il few-shot

Alcune di queste versioni non sono neutrali, e vale la pena segnalarlo qui perché il Capitolo 12 documenta i guasti che ne sono derivati.

Unsloth. Fornisce kernel ottimizzati e un percorso di caricamento a 4 bit. Il suo import è però costoso e con effetti collaterali globali: va eseguito *dopo* le validazioni di configurazione, non prima. Un bug reale documentato nella sezione 12.4.6 consisteva esattamente in una validazione posta dopo l'import, con il risultato che un errore di configurazione banale si manifestava solo dopo minuti di caricamento.

Transformers 5.3.0. Ha introdotto un cambiamento di firma in una funzione interna di rilevamento dei pacchetti, che ha rotto l'import di `trl.trainer.grpo_trainer` (sezione 12.4.3).

TRL 0.24.0. I valori predefiniti del **GRPOTrainer** in questa versione determinano quale variante dell'obiettivo GRPO viene effettivamente ottimizzata. Poiché nessuna configurazione storica del progetto li impostava esplicitamente, gli esperimenti hanno usato i default della libreria per omissione e non per scelta. L'analisi completa è nella sezione 6.3.

4.4 Il formato del prompt

Il modello riceve un'istruzione e produce il gloss. Sono state usate due modalità.

4.4.1 Zero-shot

Il prompt contiene solo l'istruzione e la frase da tradurre. La lunghezza massima del prompt è fissata a 256 token.

4.4.2 Few-shot con retrieval

Il prompt contiene, prima della frase da tradurre, $k = 3$ esempi *recuperati* dal training set in base alla somiglianza semantica con la frase corrente. Il recupero è realizzato con **sentence-transformers**: le frasi di training sono codificate in vettori una volta sola, e a inferenza si cercano i tre vicini più prossimi.

La lunghezza massima del prompt in questa modalità è 768 token, per far posto agli esempi. Questo vincolo è verificato automaticamente: `tests/validate_configs.py` rifiuta qualunque configurazione che attivi il retrieval con `max_prompt_length` inferiore a 512, perché un prompt troncato eliminerebbe silenziosamente gli esempi rendendo la cella indistinguibile dallo zero-shot.

4.4.3 Perché questa distinzione è centrale

Anticipando il Capitolo 11: la differenza fra zero-shot e few-shot è, numericamente, il fattore *più grande* osservato in tutto l'esperimento. Il modello base con vincolo di decodifica passa da ROUGE-L 0,1373 (zero-shot) a 0,4664 (few-shot), un salto di 0,33 ottenuto *senza aggiornare un solo parametro*. Il totale di variazione fra il punto di partenza peggiore e il miglior risultato con reinforcement learning è circa 0,42. Ne segue che la maggior parte dell'effetto attribuibile alla “pipeline” è in realtà effetto di prompting.

Questa è una confusione di fattori nel disegno originale: le celle che combinavano metodo di addestramento e modalità di prompting non permettevano di separare i due contributi. La correzione proposta nel Capitolo 14 è di rendere l'esperimento primario una griglia prompting $\times$ decodifica, con il metodo di addestramento come fattore successivo.

4.5 Il vocabolario di uscita

Il modello genera token, ma l'uscita valida è una sequenza di gloss. Il collegamento fra i due livelli è il vocabolario gloss di 15 472 elementi descritto nel Capitolo 3, tradotto in un insieme di 3 967 identificatori di token ammessi come *primo* token di un gloss.

La costruzione, la doppia radice necessaria per gestire il token con spazio iniziale, e le quattro forme gloss che risultano non emettibili sono trattate nel Capitolo 8.

Capitolo 5

Supervised fine-tuning

5.1 L’obiettivo, in concreto

L’addestramento supervisionato minimizza l’entropia incrociata (2.5) sulle sole posizioni della *completion*, cioè del gloss. Le posizioni del prompt sono escluse assegnando loro l’etichetta convenzionale -100 , che le librerie impiegate interpretano come “non contribuire alla loss”.

Questa distinzione è più importante di quanto sembri. Se il prompt non venisse mascherato, il modello dedicherebbe capacità a riprodurre l’inglese di ingresso, che è già noto e non serve a nulla. Peggio: su questo corpus, dove il gloss è in larga parte una copia maiuscolizzata del testo, addestrare il modello a riprodurre il testo rinforzerebbe esattamente la scorciatoia che si vuole misurare separatamente.

Formalmente, detto $m_t \in \{0, 1\}$ l’indicatore “la posizione t appartiene alla completion”, la loss effettivamente ottimizzata è

$$\mathcal{L}_{\text{SFT}}(\theta) = - \frac{\sum_i \sum_t m_t^{(i)} \log \pi_\theta(y_t^{(i)} | x^{(i)}, y_{<t}^{(i)})}{\sum_i \sum_t m_t^{(i)}}. \quad (5.1)$$

5.2 Configurazione

L’implementazione è in `src/training/sft_train.py`, funzione `run_sft`, che avvolge `trl.SFTTrainer`.

Iperparametro	Valore
Epoche	3
Batch per dispositivo	4
Accumulo del gradiente	4
Batch effettivo	16
Learning rate	$2 \cdot 10^{-5}$
Passi di warmup	50
Lunghezza massima di sequenza	768
Holdout interno per la validazione	2%
Pazienza per l’arresto anticipato	3

Il *warmup* porta il learning rate da zero al valore nominale nei primi 50 passi, evitando che i primi aggiornamenti, calcolati su gradienti ancora disallineati, danneggino i pesi. L’*holdout* del 2% è ritagliato dal training set (non dal test) e serve solo a decidere l’arresto anticipato: se la loss di validazione non migliora per 3 valutazioni consecutive, l’addestramento si ferma.

Tre epoche su 72 979 esempi con batch effettivo 16 corrispondono a circa 13 600 passi di ottimizzazione. Questo numero va tenuto a mente per il confronto con il GRPO del capitolo successivo, che ne compie 2 000.

5.3 Risultati

Valutazione su 2 000 prompt del test set, con campionamento a temperatura 0,7 e $N = 5$ generazioni per prompt (metriche versione 2, si veda il Capitolo 10).

Metrica	SFT
ROUGE-L	0,9752
Exact match	0,8130
Gloss F1	0,9765
BLEU (a frase)	0,9436
chrF	97,31
Validity	0,9998
Non-copy-token accuracy	0,9660

Nudi, questi numeri sembrano un successo pieno. Vanno letti accanto alla baseline a regole.

5.4 Il confronto che conta: SFT contro regola

Metrica	Regola	SFT	Delta
ROUGE-L	0,9685	0,9752	+0,0067
Exact match	0,5865	0,8130	+0,2265
Non-copy-token accuracy	0,9424	0,9660	+0,0236

Gli intervalli di confidenza al 95%, ottenuti con bootstrap clusterizzato per prompt (si veda il Capitolo 10), sono:

Delta	Intervallo di confidenza 95%
ROUGE-L: +0,0067	[+0,0038, +0,0098]
Exact match: +0,2265	[+0,2025, +0,2510]

Entrambi gli intervalli escludono lo zero: il vantaggio dell'SFT è statisticamente solido. Ma la sua *grandezza* dipende radicalmente dalla metrica:

- su ROUGE-L il guadagno è +0,0067, cioè lo 0,7% assoluto. Un modello addestrato su 73 mila esempi guadagna meno di un punto percentuale su una tabella di corrispondenze;
- su exact match il guadagno è +0,2265, cioè un abbattimento del tasso di errore dal 41,4% al 18,8%. In termini di errore relativo è una riduzione di circa il 55%.

Questa discrepanza è la prima manifestazione concreta della saturazione: ROUGE-L, essendo una metrica di sovrapposizione, non ha risoluzione residua per distinguere i due sistemi, mentre l'exact match sì. È un argomento operativo per promuovere l'exact match e la non-copy-token accuracy a metriche primarie (Capitolo 10).

5.5 Che cosa impara l'SFT che la regola non sa

Il confronto aggregato dice *quanto*, non *che cosa*. È stata quindi fatta un'analisi caso per caso sui 2000 prompt.

Esito	Casi
SFT corretto, regola sbagliata	586 (29,3%)
di cui: cancellazione contestuale della copula BE	382 (65% dei 586)

Cioè: due terzi del vantaggio dell'SFT sull'exact match si riducono a una sola regolarità linguistica. La copula inglese (*is*, *are*, *be*) talvolta compare nel gloss come BE e talvolta viene omessa, e l'omissione dipende dal contesto. La baseline, essendo context-free, deve scegliere una volta per tutte e sbaglia sistematicamente sull'altro caso.

5.5.1 Il controllo: estendere la regola non funziona

L'ipotesi naturale è che si possa recuperare quel vantaggio estendendo la baseline al contesto locale, cioè apprendendo una tabella su bigrammi invece che su parole singole. È stato provato:

Baseline	Exact match
Lessico su unigrammi (originale)	0,5865
Lessico esteso ai bigrammi	0,5235

Il risultato *peggiora* di 0,063. La spiegazione è la sparsità: passando ai bigrammi, molte coppie osservate nel test non compaiono nel training, e la tabella deve ricadere su un valore di riserva più spesso di quanto guadagni in precisione contestuale.

Questo controllo è metodologicamente importante perché delimita il contributo dell'SFT in modo non banale: l'SFT non sta solo memorizzando una tabella più grande, sta generalizzando una dipendenza contestuale che un conteggio diretto sui bigrammi non riesce a stimare con i dati disponibili. È il guadagno reale dell'apprendimento su questo corpus, ed è piccolo ma identificabile.

5.6 L'SFT è al tetto del corpus

Il ROUGE-L dell'SFT è 0,9752. Il miglior valore pubblicato in letteratura su ASLG-PC12 è 95,87 [24], cioè 0,9587.

Segue che l'SFT di un modello da 0,5 miliardi di parametri con adattatori LoRA *supera* lo stato dell'arte pubblicato su questo corpus. Questo non è un risultato di cui vantarsi: è la prova più chiara che la metrica non misura ciò che si crede misuri. Un modello piccolo, addestrato per tre epoche su una singola GPU, non batte anni di ricerca; batte una metrica satura.

Ne segue anche la conseguenza operativa più importante per il resto della relazione: **non c'è spazio di miglioramento residuo che il reinforcement learning possa colmare**. Con exact match 0,8130 e validity 0,9998, un metodo di rinforzo partirebbe da una policy già quasi ottima rispetto alla reward disponibile, in una regione dove i gruppi di candidati tendono a essere omogenei e quindi il vantaggio (2.13) tende a annullarsi. È esattamente ciò che si osserva, ed è quantificato nel capitolo successivo.

Capitolo 6

GRPO: apprendimento per rinforzo di gruppo

6.1 Dall'attore-critico al gruppo

Come stabilito nella (2.13), ogni metodo di gradiente di policy richiede una *baseline* $b(x)$ per ridurre la varianza. PPO [29] la ottiene addestrando una seconda rete, il *critic*, che stima il valore atteso della reward a partire dallo stato. Questo raddoppia approssimativamente il costo di memoria e introduce un secondo problema di ottimizzazione.

GRPO (*Group Relative Policy Optimization*, [31, 7]) rimuove il critic con un'idea semplice: se per lo stesso prompt x si generano G candidati, la media delle loro reward è già una stima della baseline. Formalmente, dati i candidati $o_1, \dots, o_G \sim \pi_\theta(\cdot | x)$ con reward $R_1, \dots, R_G$:

$$A_i = R_i - \frac{1}{G} \sum_{j=1}^G R_j \quad (\text{opzionalmente diviso per } \text{std}(R_1, \dots, R_G)). \quad (6.1)$$

Il gruppo è la baseline. È questa la differenza sostanziale da PPO: non c'è nessuna rete di valore da addestrare, e il segnale di apprendimento è puramente *relativo* all'interno del gruppo.

L'obiettivo completo, includendo la penalità KL (2.14), si scrive

$$J_{\text{GRPO}}(\theta) = \mathbb{E} \left[\frac{1}{Z} \sum_{i=1}^G \sum_{t=1}^{|o_i|} \rho_{i,t}(\theta) A_i \right] - \beta D_{\text{KL}}(\pi_\theta \| \pi_{\text{ref}}), \quad (6.2)$$

dove $\rho_{i,t}$ è il rapporto di probabilità fra policy corrente e policy di campionamento (con eventuale clipping in stile PPO) e Z è un *denominatore di normalizzazione* la cui scelta, come si vedrà nella sezione 6.3, non è affatto un dettaglio.

6.1.1 La conseguenza immediata: i gruppi morti

Dalla (6.1) segue un fatto elementare e decisivo. Se tutti i candidati di un gruppo ricevono la stessa reward, allora $R_i = \bar{R}$ per ogni i , dunque $A_i = 0$ per ogni i , dunque il contributo di quel prompt al gradiente è *esattamente nullo*.

Chiameremo:

gruppo a varianza nulla un gruppo in cui $\text{std}(R) = 0$;

gruppo morto un gruppo in cui tutte le reward sono al valore minimo possibile (tipicamente -1), che è un caso particolare del precedente ma qualitativamente peggiore: non solo

non c'è segnale, ma la policy non ha nemmeno un candidato parzialmente buono da cui partire;

supporto la frazione di gruppi con $\text{std}(R) > 0$, cioè la frazione di prompt che contribuiscono effettivamente all'apprendimento.

Il supporto è la grandezza diagnostica più importante per decidere se un esperimento di reinforcement learning sia *possibile* prima ancora di chiedersi se sia utile. Se il supporto è vicino a zero, l'algoritmo gira, consuma tempo di calcolo, produce metriche, ma non apprende. La misura del supporto in questo progetto è nella sezione 6.4.

6.2 Configurazione impiegata

Iperparametro	Valore
Candidati per gruppo G	8
Passi di ottimizzazione	2 000
Learning rate	$3 \cdot 10^{-6}$
Passi di warmup	200
Coefficiente KL β	0,04
Temperatura di campionamento τ	0,7
Lunghezza massima della completion	128
Lunghezza massima del prompt (zero-shot)	256
Lunghezza massima del prompt (few-shot)	768
Batch per dispositivo	1
Accumulo del gradiente	8

Il learning rate è quasi un ordine di grandezza più basso di quello dell'SFT ($3 \cdot 10^{-6}$ contro $2 \cdot 10^{-5}$): è la scelta convenzionale nel reinforcement learning su modelli linguistici, dove aggiornamenti aggressivi degradano rapidamente la policy. I resoconti su sistemi di grande scala addestrati con rinforzo [13] confermano che il regime di apprendimento va tenuto conservativo, e che il fattore limitante è tipicamente la qualità del segnale piuttosto che la dimensione del passo.

6.2.1 Un dato di contesto: l'esposizione ai dati

2 000 passi con batch 1 e accumulo 8 significano circa 16 000 prompt visti. Su un training set di 72 979 esempi corrispondono a

$$\frac{2000 \times 1 \times 8}{72979} \approx 0,22 \text{ epoche.} \quad (6.3)$$

L'SFT vede i dati tre volte; il GRPO circa un quinto di volta. Questo va detto esplicitamente perché rende improprio qualunque confronto diretto "SFT contro GRPO" interpretato come confronto fra metodi: i due bracci non hanno la stessa esposizione ai dati. Il GRPO non è stato sotto-addestrato per errore, ma per vincolo di budget di calcolo: ogni passo richiede $G = 8$ generazioni complete, quindi un passo di GRPO costa circa un ordine di grandezza più di un passo di SFT.

6.3 I denominatori: quale obiettivo si sta ottimizzando

Il termine $\mathcal{Z}$ nella (6.2) sembra un dettaglio di implementazione. Non lo è: determina come le sequenze di lunghezza diversa vengono pesate fra loro, e quindi se l'obiettivo introduca

o no un bias sulla lunghezza. È il punto su cui interviene Dr. GRPO [15], che identifica precisamente nella normalizzazione per lunghezza una fonte di bias sistematico.

TRL 0,24,0 implementa quattro varianti, selezionabili tramite un parametro di configurazione. Riportiamo la corrispondenza con il codice sorgente (`trl/trainer/grpo_trainer.py`) perché è l'unico modo di sapere con certezza che cosa sia stato ottimizzato.

Variante	Denominatore $\mathcal{Z}$	Righe
<code>grpo</code>	$ o_i $, per sequenza	1730–1731
<code>bnpo</code>	numero di token nel batch locale	1733–1734
<code>dr_grpo</code>	costante $B \times \text{max_completion_length}$	1736–1737
<code>dapo</code>	numero di token nel batch globale	1739–1741

La lettura è la seguente:

- `grpo` normalizza ogni sequenza per la propria lunghezza. Una sequenza corta con vantaggio positivo riceve un peso per token maggiore di una lunga: è il bias che Dr. GRPO critica;
- `dr_grpo` usa un denominatore *costante*, quindi tutte le sequenze contribuiscono in proporzione al proprio numero di token e non c'è riscalatura implicita;
- `bnpo` e `dapo` normalizzano sul conteggio di token del batch, rispettivamente locale e globale (aggregato fra processi). Con un solo processo le due coincidono numericamente.

6.3.1 Che cosa è stato effettivamente eseguito

I valori predefiniti di TRL 0,24,0 sono:

Parametro	Valore predefinito
Tipo di loss	<code>dapo</code>
Normalizzazione delle reward	<code>group</code>
Scalatura delle reward	<code>False</code>

Nessuna configurazione storica del progetto impostava questi parametri. Segue che tutti gli esperimenti di reinforcement learning hanno ottimizzato la variante **dapo per omissione, non per scelta**. Questo è un difetto di disegno che va dichiarato: il progetto si proponeva di studiare la variante Dr. GRPO come confronto, ma non l'ha mai attivata, perché il parametro corrispondente non era esposto nelle configurazioni.

La correzione consiste nell'espore esplicitamente questi knob (raccolti nella funzione `_grpo_objective_kwargs` di `src/training/grpo_t2g_train.py`) e nel validarli in `tests/validate_configs`, cosicché una cella che intende usare `dr_grpo` debba dichiararlo e una cella che non lo dichiara documenti il default che sta usando.

6.3.2 Un dettaglio semantico che si presta a fraintendimenti

Il parametro di scalatura delle reward ha un nome fuorviante. Impostare `scale_rewards='none'` *non* disattiva la centratura: la sottrazione della media avviene comunque (riga 1493 del sorgente), e ciò che viene omesso è soltanto la divisione per la deviazione standard (righe 1508–1509).

Non è un cavillo. Chi legge `'none'` come “lascia le reward grezze” sbaglia il modello mentale dell'algoritmo: la centratura per gruppo è la *sostanza* del GRPO, non una normalizzazione accessoria, e non è disattivabile da quel parametro. Sottrarre la media è precisamente ciò che implementa la (6.1).

6.3.3 Che cosa TRL 0.24.0 non implementa

Due meccanismi noti in letteratura non sono disponibili in questa versione, e questo delimita ciò che si può affermare.

Dynamic sampling (DAPO). DAPO [40] propone di *scartare* i gruppi a varianza nulla e ricampionare, così da mantenere il supporto alto. TRL 0,24,0 non lo implementa: si limita a registrare la metrica `frac_reward_zero_std`. Il problema dei gruppi morti è quindi *osservabile* ma non *mitigato*.

Clip-higher. La tecnica di asimmetrizzazione della finestra di clipping richiede di impostare un limite superiore distinto (`epsilon_high`), che va specificato esplicitamente e non era specificato.

Chi legge questa relazione deve quindi intendere “GRPO” come “GRPO nella variante `dapo` di normalizzazione, senza dynamic sampling e senza clip-higher”. In particolare, nessuna delle mitigazioni che la letteratura propone contro il collasso del supporto era attiva.

6.4 La misura decisiva: il supporto dei rollout

Prima di chiedersi se il GRPO migliori le metriche, si può chiedere se abbia *materiale* su cui lavorare. La misura è diretta: si generano G candidati per un campione di prompt, si calcolano le reward, e si osservano la reward media, la frazione di gruppi al valore minimo (*floor*), la frazione a varianza nulla e la frazione di gruppi morti.

Substrato	Reward media	Floor	Var. nulla	Gruppi morti
Base zero-shot, con Trie	−0,8034	0,5387	0,2145	0,1990
Base zero-shot, senza Trie	−0,9579	0,9054	0,7460	0,7390
Base few-shot, con Trie	−0,3233	0,0982	0,0780	0,0085
Policy SFT	+0,9462	0,0000	0,8480	—

Questa tabella è il cuore analitico della relazione. Va letta riga per riga.

Base zero-shot senza vincolo. Reward media −0,9579, quasi il minimo possibile. Il 90,5% dei gruppi è interamente al valore minimo e il 73,9% è morto. In questa condizione il GRPO *non può funzionare*: tre gruppi su quattro non producono gradiente. Non è una questione di iperparametri o di pazienza: manca il segnale.

Base zero-shot con vincolo. Il Trie porta i gruppi morti dal 73,9% al 19,9%. Il vincolo simbolico *crea* il supporto, forzando i candidati dentro la regione in cui la reward è discriminante. Questo è il risultato neuro-simbolico non ovvio annunciato nell’introduzione, ed è approfondito nel Capitolo 8.

Base few-shot con vincolo. Il substrato migliore: reward media −0,3233, gruppi morti allo 0,85%. Combinare prompting e vincolo rende l’esperimento di reinforcement learning tecnicamente sensato.

Policy SFT. Il caso opposto e più istruttivo. La reward media è +0,9462: la policy è già quasi ottima. Non ci sono gruppi al valore minimo. Ma l’84,8% dei gruppi ha *varianza nulla*, perché tutti gli otto candidati sono corretti e ricevono la stessa reward massima. Il gradiente è nullo per la ragione opposta a prima: non per fallimento, ma per saturazione.

6.4.1 La conclusione meccanica

Le due estremità della tabella mostrano che il GRPO su questo compito è schiacciato fra due fallimenti simmetrici:

$$\underbrace{\text{policy troppo debole}}_{\text{gruppi morti: nessun candidato buono}} \quad \Big| \quad \underbrace{\text{policy troppo forte}}_{\text{varianza nulla: tutti i candidati buoni}} \quad (6.4)$$

Nel primo caso $A_i = 0$ perché $R_i = R_{\min}$ per ogni i ; nel secondo perché $R_i = R_{\max}$ per ogni i . La finestra utile, in cui il gruppo contiene sia candidati buoni sia cattivi, è stretta e su questo corpus quasi inesistente, perché il compito è quasi deterministico (sezione 3.2.3).

Vale la pena osservare che la mitigazione apparentemente ovvia — concentrare l’addestramento sugli esempi difficili, dove la varianza dovrebbe essere maggiore — è stata studiata su modelli linguistici piccoli messi a punto con GRPO e produce rendimenti decrescenti [36]. Gli esempi difficili tendono a spostarsi dalla finestra utile verso il regime dei gruppi morti, dove nessun candidato è buono, invece di restare nella regione in cui i candidati differiscono.

Questo è il *meccanismo* del risultato negativo. Non si tratta di dire “il GRPO non ha funzionato”; si tratta di mostrare, con numeri, che sull’84,8% dei gruppi il gradiente era esattamente zero dopo l’SFT, e sul 73,9% era esattamente zero prima di qualunque addestramento senza vincolo.

6.5 Il curriculum e il suo generatore di numeri casuali

Una delle mitigazioni tentate è un *curriculum*: filtrare gli esempi per difficoltà crescente lungo l’addestramento, in modo che la policy affronti prima i casi in cui è probabile che produca almeno un candidato buono. È implementato dalla classe `CurriculumFilteredDataset`.

L’implementazione originale usava il generatore di numeri casuali *globale* del modulo `random`. Questo introduce un difetto di riproducibilità sottile: lo stato del generatore globale dipende da quante volte è stato consumato altrove nel processo (dal caricamento del modello, dal campionamento, da qualunque componente che usi `random`). Il risultato è che gli esempi selezionati dal curriculum cambiano se cambia il consumo del generatore in una parte del tutto indipendente del codice.

La correzione applicata è di dotare il curriculum di un generatore dedicato, derivato deterministicamente dal seed e dallo stadio:

$$\text{rng} = \text{random.Random}(\text{seed} \times 1000 + \text{stage}). \quad (6.5)$$

Il seed viene passato dal punto di chiamata in `src/training/grpo_t2g_train.py`. In questo modo la sequenza di esempi di ogni stadio è funzione soltanto di `(seed, stage)`.

Questo tipo di correzione non cambia i risultati riportati, ma cambia il loro statuto: prima erano non riproducibili in modo non dichiarato, ora lo sono.

Capitolo 7

Il disegno delle reward

7.1 Il problema di progettazione

Nell’addestramento supervisionato l’obiettivo è dato: la (2.5) non richiede scelte. Nel reinforcement learning invece la funzione di reward è il progetto: definisce che cosa il modello considererà buono, e ogni sua imperfezione verrà sfruttata dall’ottimizzazione. È il fenomeno del *reward hacking* [1, 4], di cui la deriva verso risposte più lunghe è l’esempio più documentato [32].

La convenzione adottata in questo progetto è che ogni componente di reward produca un valore in $[0, 1]$, che viene poi mappato in $[-1, 1]$ tramite

$$R = 2s - 1, \quad s \in [0, 1]. \quad (7.1)$$

La ragione della mappatura è che il vantaggio (6.1) è invariante per traslazione, ma un intervallo centrato rende più leggibile la telemetria: un valore negativo indica “sotto la metà della qualità massima”.

L’implementazione è raccolta in `src/rewards/t2g_rewards.py`.

7.2 Lo stack di sette componenti

La reward complessiva è una combinazione convessa di sette componenti, con pesi che sommano esattamente a 1,0. La somma unitaria non è un vezzo: è verificata automaticamente da `tests/validate_configs.py`, che rifiuta qualunque configurazione in cui i pesi non sommino a uno, per evitare che la scala della reward cambi silenziosamente fra celle sperimentali.

Componente	Che cosa misura	Peso
<code>translation</code>	somiglianza con il gloss di riferimento	0,20
<code>bleu</code>	sovrapposizione di n -grammi	0,20
<code>gold_structure</code>	aderenza alla struttura del riferimento	0,20
<code>gloss_order</code>	correttezza dell’ordine dei gloss	0,10
<code>verifier_scaled</code>	esito di un verificatore esterno	0,10
<code>format</code>	conformità al formato di uscita	0,10
<code>repetition</code>	assenza di ripetizioni degeneri	0,10
Totale		1,00

7.3 La telemetria: quali componenti portano informazione

Durante l'addestramento GRPO (cella a partire dal modello base) è stato registrato il valore medio di ciascuna componente. I valori sono nella scala $[-1, 1]$ della (7.1).

Componente	Valore medio osservato
<code>translation</code>	+0,215
<code>bleu</code>	-0,359
<code>gold_structure</code>	+0,086
<code>gloss_order</code>	+0,085
<code>verifier_scaled</code>	-0,291
<code>format</code>	+0,998
<code>repetition</code>	+0,966

Le ultime due righe sono il punto. Con valori +0,998 e +0,966 su una scala il cui massimo è +1, le componenti `format` e `repetition` sono **sature**: praticamente ogni candidato le soddisfa.

La conseguenza è meccanica e segue dalla (6.1). Una componente satura ha varianza quasi nulla *all'interno del gruppo*, dunque contribuisce al vantaggio di ogni candidato in misura quasi identica, dunque il suo contributo si cancella nella sottrazione della media. Il 20% del peso complessivo della reward non produce alcun segnale di apprendimento: agisce come una costante additiva.

Questa non è una critica al disegno originale, che aveva senso come rete di sicurezza (impedire che il modello degeneri in ripetizioni o violi il formato). È un'osservazione sull'*allocazione dell'informazione*: se il 20% del budget di reward è speso su vincoli che non vengono mai violati, il segnale effettivo viene dalle altre cinque componenti.

7.4 Tre componenti rimosse, e perché

Lo stack storico conteneva tre componenti aggiuntive, che sono state rimosse dopo averne dimostrato la ridondanza. La rimozione è documentata qui perché il *modo* in cui è stata giustificata è metodologicamente il contributo più solido di questo capitolo.

Le componenti erano `structural_dense`, `viterbi_distance` e `soft_viterbi_distance`. Tutte tre miravano a misurare la plausibilità strutturale della sequenza generata rispetto a un modello di transizione fra gloss.

7.4.1 Argomento algebrico

Le tre componenti sono definite come normalizzazioni di una quantità di “energia” del percorso rispetto a un limite superiore che dipende dalla lunghezza della sequenza. Quando generato e riferimento hanno la *stessa lunghezza*, quel limite compare sia al numeratore sia al denominatore e si cancella algebricamente. Le tre formule collassano allora sullo stesso segnale: non sono tre misure indipendenti, ma tre riparametrizzazioni della medesima.

7.4.2 Verifica empirica

L'argomento algebrico vale a lunghezza uguale, che sul corpus copre soltanto il 27,5% delle righe (sezione 3.3). Serviva dunque una verifica empirica. È stata eseguita una valutazione di controllo: tutte le configurazioni con e senza le tre componenti sono rimaste entro $\pm 0,006$ dal valore di controllo 0,5114.

Cioè: aggiungere o togliere tre componenti di reward non sposta la metrica oltre il rumore. Le componenti sono state quindi rimosse, insieme a circa 8 774 righe di codice relative all'infrastruttura che le supportava (implementazione di automi a stack, calcolo di percorsi di Viterbi, macchinari di supporto).

Va segnalato che il calcolo dei percorsi di Viterbi *sopravvive* nel progetto, ma con statuto diverso: come strumento *diagnostico* in `src/analysis/markov_diagnostics.py`, non come reward. È una distinzione importante: una quantità può essere informativa da ispezionare e inutile da ottimizzare.

7.5 Una componente nuova: edit_validity

Al posto delle tre rimosse è stata progettata una singola componente, con l'obiettivo di catturare due proprietà con una sola misura: la vicinanza al riferimento e l'appartenenza al vocabolario ammesso.

7.5.1 Definizione

$$\text{edit_sim} = 1 - \frac{\text{lev}(g, r)}{\max(|g|, |r|)}, \quad (7.2)$$

$$s = (1 - w) \cdot \text{edit_sim} + w \cdot \text{frac_in_vocab}, \quad (7.3)$$

$$R = 2s - 1, \quad (7.4)$$

dove lev è la distanza di Levenshtein a livello di token fra generato g e riferimento r , frac_in_vocab è la frazione di token generati appartenenti al vocabolario gloss, e $w \in [0, 1]$ è un peso con valore predefinito 0,5.

La normalizzazione per $\max(|g|, |r|)$ nella (7.2) è la scelta che rende la misura ben definita anche quando le lunghezze differiscono, ed è esattamente il punto su cui è stata condotta l'analisi avversaria che segue.

7.5.2 Test avversario 1: la reward premia la verbosità?

L'ipotesi da escludere è la più classica: che il modello possa aumentare la reward allungando l'uscita [32]. La verifica è analitica. Supponiamo che il generato coincida con il riferimento e vi si appendano k token spuri. Allora $\text{lev} = k$, $|g| = G + k$ con $G = |r|$, e dunque

$$\text{edit_sim} = 1 - \frac{k}{G + k}, \quad (7.5)$$

che è *strettamente decrescente* in k . La misura empirica conferma una perdita di circa $-0,13$ per token appeso.

L'ipotesi è dunque refutata. Va detto esplicitamente che questa era l'ipotesi di partenza dell'analisi, formulata come sospetto prima della verifica: la verbosità non è un canale di attacco per questa reward. Documentare la ritrattazione, e non solo la conclusione, è parte del contributo.

7.5.3 Test avversario 2: la calibrazione di w

Il peso w nella (7.3) bilancia due obiettivi che possono entrare in conflitto. Se w è troppo alto, la reward premia l'uso di token in vocabolario indipendentemente dal fatto che formino una sequenza sensata. Il test costruisce tre candidati avversari e verifica l'*ordinamento* che la reward induce fra loro.

Candidato	R con $w = 0,5$	R con $w = 0,75$
Sequenza priva di senso (<i>garbage</i>)	0,000	—
Ripetizione degenerare di token in vocabolario	+0,125	+0,563
Quasi corretto, con un token fuori vocabolario	+0,500	+0,500

Con $w = 0,5$ l'ordinamento è corretto: il candidato quasi corretto (+0,500) batte la ripetizione degenerare (+0,125).

Con $w = 0,75$ l'ordinamento **si inverte**: la ripetizione degenerare (+0,563) supera il candidato quasi corretto (+0,500). A quel valore di w la reward premia esplicitamente la produzione di token legali ripetuti rispetto a una traduzione quasi giusta con una parola sconosciuta.

Il valore predefinito è quindi fissato a $w = 0,5$, e `tests/validate_configs.py` impone che il peso della componente fuori vocabolario resti nell'intervallo $[0, 0,6]$: al di sopra di quella soglia esiste un attacco dimostrato.

Questo è il tipo di verifica che si può fare *prima* di consumare tempo di GPU: non richiede addestramento, solo la valutazione della reward su candidati costruiti a mano. Ha costo trascurabile e previene una classe di guasti che altrimenti si scoprirebbe solo osservando uscite degenerate dopo migliaia di passi.

7.5.4 Test avversario 3: quanto pesa il gate sul vocabolario

Un'altra preoccupazione legittima è che il termine `frac_in_vocab` diventi il fattore dominante, azzerando la reward di candidati che sarebbero altrimenti informativi. È stato quindi contato, sui gruppi valutati in condizione zero-shot con vincolo di decodifica, quanti dei valori di reward pari al minimo (-1) fossero attribuibili al gate sul vocabolario.

Causa del valore minimo	Occorrenze	Frazione
Gate sul vocabolario (token fuori vocabolario)	131/10 000	2,43%
<i>Floor</i> legittimo (candidato effettivamente cattivo)	—	97,57%

Il gate non è la causa del collasso del supporto documentato nella sezione 6.4: contribuisce per il 2,43%. Il restante 97,57% dei valori minimi corrisponde a candidati genuinamente cattivi. È una verifica necessaria, perché se il gate fosse stato la causa dominante la lettura del risultato negativo sarebbe stata diversa: non “il compito non offre segnale” ma “la nostra reward lo distrugge”.

7.6 Riepilogo del metodo

La procedura seguita per ogni componente di reward è la seguente, e vale come raccomandazione generale:

1. definire la componente in forma chiusa;
2. derivare analiticamente il suo comportamento sotto le manipolazioni ovvie (allungare, ripetere, riempire con token legali);
3. costruire a mano tre o quattro candidati avversari e verificare l'*ordinamento* indotto, non il valore assoluto;
4. cercare la soglia di iperparametro a cui l'ordinamento si rompe, e fissare il valore predefinito con margine da quella soglia;

5. codificare il vincolo in un test di validazione delle configurazioni, così che non possa essere violato per distrazione.

I punti 3 e 4 sono quelli che hanno effettivamente trovato difetti in questo progetto. Il punto 5 è quello che impedisce che i difetti tornino.

Capitolo 8

Decodifica vincolata: il componente simbolico

Questo capitolo descrive l'unico elemento propriamente *neuro-simbolico* del sistema, e riporta il risultato più controintuitivo dell'intero lavoro: il vincolo simbolico *peggiora* la metrica di sovrapposizione e contemporaneamente *abilita* il segnale di reward.

8.1 Il problema: dal vincolo sulle parole al vincolo sui token

L'uscita valida è una sequenza di gloss appartenenti a un vocabolario finito G di 15472 elementi. Il modello, però, genera *token*, e un gloss corrisponde in generale a più token (sezione sui fondamenti di tokenizzazione). Non si può quindi semplicemente restringere il vocabolario di uscita: bisogna tradurre il vincolo sul livello delle parole in un vincolo sul livello dei token.

La formulazione corretta è quella della (2.10): al passo t , dato il prefisso di token già emesso, sono ammessi i token che mantengono il prefisso estendibile a un elemento di G . Il Trie costruito su G è la struttura dati che rende questo calcolo immediato.

Operativamente il vincolo agisce sui logits come nella (2.9): si pone $z_{t,v} = -\infty$ per ogni $v \notin \mathcal{A}_t$ prima della softmax.

8.2 Implementazione

Due componenti, con responsabilità separate:

File	Ruolo
<code>src/grammar/gloss_grammar.py</code>	<code>GlossVocabularyMask</code> : il Trie e il calcolo dell'insieme ammissibile
<code>src/grammar/grammar_logits_processor.py</code>	<code>GlossVocabularyLogitsProcessor</code> : l'applicazione ai logits

La separazione è deliberata: la maschera è una struttura dati pura, testabile senza modello, mentre il processore di logits è l'adattatore verso l'interfaccia di generazione della libreria. Tutti i test sulla correttezza del vincolo agiscono sulla prima, senza caricare la GPU.

8.2.1 La doppia radice

Un dettaglio che non è un dettaglio. Il tokenizzatore codifica lo spazio come parte del token successivo: il token per "HOUSE" e quello per " HOUSE" sono identificatori distinti. Ne segue che il primo gloss di una sequenza e i gloss successivi hanno insiemi di primi token *diversi*.

Il Trie è quindi costruito con **due radici**:

`root` usata per il primo gloss della sequenza, i cui token iniziali non hanno spazio davanti;

`space_root` usata per tutti i gloss successivi, i cui token iniziali portano lo spazio.

Se si usasse una sola radice si aprirebbe un difetto concreto: il modello potrebbe concatenare due gloss senza separatore, producendo forme come **DEBUTRECHT** (dalla giustapposizione di **DEBUT** e **RECHT**) che il Trie a radice singola accetterebbe come prefisso valido perché non distingue il confine di parola. La doppia radice rende il confine esplicito nella struttura.

8.2.2 Numeri della costruzione

Grandezza	Valore
Gloss nel vocabolario	15 472
Identificatori di token ammessi come primo token	3 967
Gloss con il primo token bloccato	4
Massa di probabilità irraggiungibile	12/897 971

I quattro gloss non emettibili sono **-RRB-**, **WWW.8HOURS.EU**, una forma costituita da un asterisco isolato, e una forma che inizia con il carattere di *byte order mark*. Sono tutti artefatti del corpus (sezione 3.3), non gloss linguisticamente significativi: la loro inaccessibilità non costituisce una perdita.

La massa irraggiungibile di 12 su 897 971 è la verifica quantitativa che il vincolo non stia tagliando fuori qualcosa di rilevante: circa una parte su settantacinquemila.

8.2.3 L'interfaccia per gli usi non generativi

Il vincolo serve anche al di fuori della generazione: gli obiettivi ausiliari del Capitolo 9 hanno bisogno di conoscere l'insieme ammesso in modalità *teacher-forced*, cioè per prefissi noti in blocco e non generati uno alla volta. A questo scopo è stato aggiunto il metodo

```
allowed_mask_for_prefixes(prefixes, vocab_size, device)
```

con il supporto interno `_allowed_after_prefix`. Restituisce direttamente la maschera booleana per un insieme di prefissi, evitando di duplicare la logica del Trie in due punti del codice: la definizione di “token ammesso” esiste in un solo posto, e sia la generazione sia la loss ausiliaria la consumano.

8.3 L'alternativa scartata: l'automa a stack

Il progetto conteneva, in una fase precedente, un'implementazione di automa a stack (*pushdown automaton*) con parsing LL(1), pensata per imporre una grammatica context-free invece di un semplice lessico chiuso. È stata rimossa. Le ragioni vanno documentate, perché la rimozione di codice funzionante richiede giustificazione.

1. **Non è mai stata eseguita.** Il componente non ha mai prodotto un risultato sperimentale. Manteneva quindi un costo di manutenzione senza contropartita.
2. **Conflitto di parsing non risolto.** La grammatica presentava un conflitto fra la produzione vuota e l'insieme dei simboli che possono seguire un non terminale, innescato dai gloss che iniziano con una cifra (per esempio 0, 8, 08.30). Un parser LL(1) richiede che l'insieme dei primi simboli e quello dei simboli seguenti siano disgiunti in presenza di produzioni vuote; qui non lo erano.

3. **Sovradimensionamento.** Il linguaggio target è **gloss***: una chiusura di Kleene su un insieme finito. È un linguaggio regolare, riconoscibile da un automa a stati finiti. Un automa a stack riconosce linguaggi context-free, strettamente più espressivi: la potenza aggiuntiva non serviva a nulla su questo compito.

La conclusione generale è che la scelta del formalismo simbolico deve seguire la complessità effettiva del linguaggio da vincolare, non l’ambizione del disegno. Sulla decodifica grammaticalmente vincolata in senso pieno esiste letteratura matura [5, 23]; sarebbe la strada corretta se il vincolo fosse davvero context-free, che qui non è il caso.

8.4 Il risultato controintuitivo

Ecco l’effetto del vincolo sulla cella base zero-shot.

Metrica	Senza Trie	Con Trie
ROUGE-L	0,3648	0,1373
Non-copy-token accuracy	0,0006	0,0432
Validity	—	0,9006

Il vincolo fa *crollare* ROUGE-L da 0,3648 a 0,1373: una perdita apparente di 0,23. Allo stesso tempo la non-copy-token accuracy passa da 0,0006 a 0,0432, un fattore circa 70.

8.4.1 Come si spiega

Le due metriche misurano cose diverse, e il vincolo agisce su di esse in direzioni opposte.

Senza vincolo, il modello base fa la cosa per cui è stato pre-addestrato: produce inglese. Poiché il gloss è in larga parte inglese maiuscolizzato (sezione 3.2), produrre inglese ottiene un ROUGE-L rispettabile “per caso”. Ma la non-copy-token accuracy, che valuta solo le posizioni dove il gloss *differisce* dal testo, è 0,0006: praticamente nulla. Il modello non ha imparato niente sul gloss; sta beneficiando della sovrapposizione lessicale del corpus.

Con il vincolo, il modello è costretto a emettere solo gloss del vocabolario. Perde la possibilità di copiare l’inglese, quindi perde il ROUGE-L gratuito. In compenso, sulle posizioni che richiedono una trasformazione reale, comincia a produrre qualcosa di corretto (0,0432 invece di 0,0006).

Detto in modo compatto: il vincolo trasforma una copia con punteggio alto in un tentativo di traduzione con punteggio basso. La metrica di sovrapposizione registra un peggioramento; la metrica che conta registra un miglioramento di quasi due ordini di grandezza.

8.4.2 L’effetto veramente importante: il vincolo crea il supporto

Il ROUGE-L, però, non è la ragione per cui il vincolo è presente nel sistema. La ragione è nella tabella del supporto dei rollout (sezione 6.4), che vale la pena riportare nella parte pertinente:

Substrato zero-shot	Gruppi morti	Reward media
Senza Trie	0,7390	−0,9579
Con Trie	0,1990	−0,8034

Senza vincolo, il 74% dei gruppi è morto: come stabilito nella sezione 6.1.1, per quei prompt il gradiente è esattamente zero e il reinforcement learning non è nemmeno definito come procedura utile. Con il vincolo, i gruppi morti scendono al 20%.

Il componente simbolico non migliora le metriche: rende possibile l'apprendimento. È il risultato neuro-simbolico di questo lavoro, e ha una forma argomentativa precisa. Il vincolo restringe il supporto della distribuzione generativa alla regione in cui la reward è *discriminante*. Fuori da quella regione tutti i candidati sono ugualmente cattivi, quindi la sottrazione (6.1) li annulla tutti; dentro, differiscono, quindi producono gradiente.

Questo giustifica anche una raccomandazione operativa generale: quando si prepara un esperimento di reinforcement learning su un compito con uscita strutturata, conviene misurare il supporto dei rollout *prima* di addestrare, e considerare il vincolo simbolico non come un accessorio di qualità ma come una preconditione per la fattibilità.

8.5 Il costo

Il vincolo non è gratuito. A ogni passo di generazione richiede:

- la risoluzione del nodo corrente del Trie a partire dal prefisso;
- la costruzione di una maschera booleana sulla dimensione del vocabolario;
- l'applicazione della maschera ai logits.

Il costo dominante è la costruzione della maschera, lineare in $|\mathcal{V}|$. Nel regime di questo progetto (modello piccolo, $G = 8$ candidati per gruppo, completion di al massimo 128 token) resta trascurabile rispetto al costo del passaggio in avanti della rete. Su modelli più grandi la valutazione andrebbe rifatta, e la memorizzazione delle maschere per nodo del Trie sarebbe l'ottimizzazione naturale.

8.6 Che cosa non fa il vincolo

Per evitare sovrainterpretazioni, va delimitato ciò che il Trie garantisce.

Garantisce che ogni parola emessa appartenga al vocabolario gloss e che i confini di parola siano rispettati.

Non garantisce che la *sequenza* sia grammaticalmente valida in ASL: il linguaggio riconosciuto è **gloss***, cioè qualunque successione di gloss legali.

Non garantisce che la sequenza sia una traduzione corretta della frase di ingresso. Il vincolo è sintattico e non guarda l'ingresso.

Non impedisce le ripetizioni degeneri: **HOUSE HOUSE HOUSE** è una sequenza perfettamente ammissibile per il Trie. È il motivo per cui la reward include una componente dedicata alla ripetizione (Capitolo 7) e per cui il test avversario sul peso w della sezione precedente era necessario.

Il vincolo è dunque una condizione necessaria di buona formazione, non una condizione sufficiente di correttezza. Confondere le due cose porterebbe a attribuire al componente simbolico garanzie che non ha.

8.7 Il costo nascosto della generazione vincolata

Il risultato della sezione 8.4 — il vincolo peggiora la metrica di sovrapposizione — non è un'anomalia locale di questo progetto. È un fenomeno riconosciuto nella letteratura sulla generazione strutturata, e conviene collocarlo perché ne chiarisce la natura.

Il punto teorico è quello anticipato a proposito della (2.9): mascherare i logits e rinormalizzare produce una distribuzione che *non* è la distribuzione condizionata del modello all'insieme delle sequenze valide. La rinormalizzazione è locale, passo per passo, mentre il condizionamento corretto richiederebbe di tenere conto della probabilità che il prefisso corrente possa essere completato in una sequenza valida. Ne segue un errore sistematico, ed esistono metodi che lo correggono restituendo stimatori asintoticamente non distorti [37].

Le conseguenze pratiche di questa distorsione sono state documentate da due angoli distinti. Da un lato, il vincolo può degradare la qualità del contenuto generato anche quando garantisce la forma, con un costo che è stato caratterizzato come dipendente dal condizionamento sulla bozza [28]. Dall'altro, il vincolo può indurre il modello a proseguire lungo una struttura formalmente valida ma sostanzialmente sbagliata, un effetto descritto come *tassa di allineamento della decodifica vincolata* [42].

Nel presente lavoro la manifestazione è precisamente questa: il modello base vincolato produce sequenze di gloss ben formate (validity 0,9006) ma con punteggio di sovrapposizione basso (0,1373), mentre senza vincolo produce sequenze mal formate (validity 0,1043) con punteggio più alto (0,3648). La differenza rispetto alla letteratura citata è nella conclusione operativa: qui il vincolo va mantenuto *nonostante* il costo, perché è ciò che rende non nullo il gradiente del reinforcement learning (sezione 6.4). Il costo sulla metrica di sovrapposizione è il prezzo pagato per avere un segnale di apprendimento.

Capitolo 9

Obiettivi ausiliari

Il Capitolo 6 ha mostrato che il reinforcement learning su questo compito è schiacciato fra gruppi morti e gruppi a varianza nulla. Una risposta possibile è cercare segnale altrove: non in una reward su sequenze generate, ma in obiettivi ausiliari calcolabili in modalità *teacher-forced*, cioè sui riferimenti noti.

Sono stati implementati due obiettivi. Entrambi sono documentati qui insieme ai criteri che ne governano la promozione a esperimento vero, perché il punto metodologico del capitolo è proprio questo: **qualificare un obiettivo prima di addestrarlo**.

9.1 Obiettivo 1: la massa ammessa

9.1.1 Formulazione

L'idea è di premiare il modello per la probabilità che assegna *all'insieme* dei token legali, invece che al singolo token corretto. È esattamente la formulazione (2.6) introdotta nei fondamenti:

$$\mathcal{L}_{\text{AM}} = -\log \sum_{v \in \mathcal{A}_t} \pi_{\theta}(v \mid x, y_{<t}), \quad (9.1)$$

dove $\mathcal{A}_t$ è fornito dal Trie del Capitolo 8 tramite il metodo `allowed_mask_for_prefixes` e i prefissi $y_{<t}$ sono quelli del riferimento.

Questo è apprendimento a etichette parziali [2], equivalente alla massimizzazione della verosimiglianza marginale [8]. Va segnalato che nel caso profondo la formulazione ingenua della (9.1) è, contrariamente all'intuizione, un punto di partenza competitivo e non un ripiego [30]: è la ragione per cui non è stata introdotta alcuna variante più elaborata. La loss totale diventa

$$\mathcal{L} = \mathcal{L}_{\text{SFT}} + \lambda \mathcal{L}_{\text{AM}} \quad (9.2)$$

con λ configurabile.

L'implementazione è in `src/training/allowed_mass_loss.py`, integrata nel percorso di addestramento tramite `src/training/auxiliary_sft_trainer.py`.

9.1.2 Nota numerica

Il termine $\log \sum \exp$ implicito nella (9.1) è calcolato in precisione a 32 bit, anche quando il resto del passaggio in avanti gira in precisione ridotta. La ragione è che la somma è su un insieme di migliaia di elementi con logits di ampiezza variabile, e in precisione dimezzata la perdita di cifre significative è sufficiente a produrre gradienti sbagliati. È il tipo di dettaglio che non si nota fino a quando l'addestramento diverge senza motivo apparente.

9.1.3 Che cosa questo obiettivo può e non può fare

Qui va ripetuta con forza l'osservazione della sezione sui fondamenti: la (9.1) è **piatta** sull'insieme ammesso. Qualunque ridistribuzione di massa *interna* ad $\mathcal{A}_t$ lascia la loss invariata.

Ne segue una previsione precisa, che è anche il criterio con cui l'obiettivo va valutato:

- l'obiettivo può migliorare la **calibrazione**: quanta probabilità il modello colloca su token legali, e quindi quanta massa il vincolo di decodifica deve rimuovere;
- l'obiettivo **non può**, da solo, migliorare l'exact match, perché non fornisce alcuna preferenza fra i token legali.

Chiunque si aspettasse un miglioramento dell'exact match da questo obiettivo avrebbe un modello mentale sbagliato della sua forma funzionale. Dichiararlo in anticipo evita di interpretare un risultato nullo sull'exact match come un fallimento dell'implementazione.

9.2 Obiettivo 2: la loss strutturata

9.2.1 Formulazione

Il secondo obiettivo introduce un termine che modella le *transizioni* fra gloss consecutivi, nello spirito dei campi aleatori condizionali (CRF) e delle loro varianti applicate a compiti di etichettatura strutturata [6, 12, 41].

Il modello è un CRF *sparso a lunghezza esatta*: si assume che la sequenza di gloss abbia la stessa lunghezza della sequenza sorgente (che è il caso nel 27,5% del corpus, sezione 3.3) e si assegna un punteggio alle transizioni fra stati consecutivi.

Lo spazio degli stati è ridotto a 513 elementi: i 512 gloss più frequenti più uno stato aggregato **OTHER** che raccoglie tutti gli altri. Le transizioni sono stimate esclusivamente dal training set *dopo* la suddivisione, per evitare contaminazione.

I file coinvolti sono:

File	Ruolo
src/datasets/structured_transitions.py	stima delle transizioni dal training
src/models/structured_gloss_head.py	la testa strutturata
src/training/structured_sft.py	integrazione nella loss

9.2.2 Vincolo di memoria

Un CRF ingenuo richiederebbe un tensore di punteggi di forma $[B, T, |\mathcal{V}|]$, con B dimensione del batch, T lunghezza e $|\mathcal{V}|$ dimensione del vocabolario. Su un vocabolario di decine di migliaia di elementi questo è proibitivo.

L'implementazione evita completamente quel tensore usando accumulazione sparsa (`scatter_add`) in precisione a 32 bit: si aggregano i contributi soltanto per gli indici effettivamente coinvolti. Il requisito di memoria diventa proporzionale al numero di transizioni osservate, non al prodotto delle dimensioni.

9.2.3 Qualificazione: quanto informa realmente il gloss precedente?

Prima di spendere tempo di GPU su questo obiettivo, è stata posta la domanda diretta: *sapere il gloss precedente aggiunge informazione, dato che si conosce già il token sorgente?*

La domanda è formulabile in modo esatto con l'entropia condizionale (2.16). La stima è stata condotta su 21 917 token esclusi dall'addestramento, con una strategia di *backoff* a tre livelli

$$(\text{prev}, \text{src}) \rightarrow \text{src} \rightarrow \text{unigramma} \quad (9.3)$$

per gestire le combinazioni non osservate.

Quantità	Valore
$H(\text{gloss} \mid \text{src})$	0,8560 bit
$H(\text{gloss} \mid \text{src}, \text{prev})$	0,8116 bit
Guadagno	+0,0444 bit
Frazione dell'incertezza residua rimossa	5,19%

Il gloss precedente aggiunge 0,0444 bit, cioè rimuove circa il 5% dell'incertezza che resta dopo aver visto il token sorgente. Non è zero: esiste un prior strutturale reale. Ma è piccolo, e fissa un limite superiore realistico a quanto un termine di transizione possa contribuire.

9.2.4 Il caveat, dichiarato

La stima soffre di una limitazione che va esposta e non nascosta: è calcolata solo sulle coppie *allineate*, cioè sul 27,5% del corpus in cui testo e gloss hanno la stessa lunghezza.

Questo è problematico in modo specifico. Il fenomeno linguistico più rilevante identificato nell'analisi dell'SFT (sezione 5.5) è la cancellazione contestuale della copula **BE**, responsabile del 65% dei casi in cui l'SFT batte la baseline a regole. Ma una cancellazione *cambia la lunghezza*: quei casi stanno per costruzione fuori dal sottoinsieme allineato.

Il valore +0,0444 bit è dunque una stima ottenuta dove il fenomeno di interesse non c'è. Potrebbe sottostimare o sovrastimare il vero contributo. È una limitazione del metodo di stima, non un difetto dell'implementazione, e la sua risoluzione richiederebbe un allineamento non monotono fra le due sequenze.

9.2.5 Onestà sulle ritrattazioni

Questa stima è stata corretta due volte prima di arrivare al valore riportato. Le versioni precedenti erano affette da contaminazione fra training e valutazione e da un backoff mal specificato che assorbiva parte del guadagno apparente. Il valore +0,0444 bit è quello sopravvissuto alla verifica.

Segnalarlo non è autodenigrazione: è la condizione perché il numero sia credibile. Un guadagno di 0,0444 bit è abbastanza piccolo da poter essere prodotto interamente da un errore di metodo, e chi legge deve sapere che l'errore è stato cercato.

9.3 Il contratto di non regressione

Aggiungere termini alla loss di addestramento è una modifica invasiva: rischia di alterare il comportamento anche quando è disattivata. Il progetto adotta quindi un *contratto* verificato da test, il cui contenuto è il seguente.

Inerzia esatta a peso nullo. Con peso 0 il percorso di addestramento deve essere **bit-identico** a quello del **SFTTrainer** originale. Non “approssimativamente uguale”: identico. La verifica è automatica, e serve a garantire che tutte le celle sperimentali che non usano gli obiettivi ausiliari non ne siano influenzate.

Validazione prima del modello. La funzione `resolve_auxiliary_config` valida la configurazione *prima* che il modello venga caricato. Un errore di configurazione deve costare secondi, non minuti (si veda la sezione 12.4.6 per il caso reale in cui questa regola era violata).

Incompatibilità rifiutate esplicitamente. Le opzioni di *packing* (concatenazione di più esempi in una sequenza) e *padding-free* sono rifiutate, perché entrambe distruggono la corrispondenza fra posizione nel tensore e posizione nella sequenza originale, da cui gli obiettivi ausiliari dipendono. Rifiutare è preferibile a produrre silenziosamente risultati sbagliati.

Un solo processo. La funzione `require_single_process` rifiuta l'esecuzione con più di un processo. La ragione è che l'aggregazione delle statistiche strutturate non è stata verificata in regime distribuito; piuttosto che dedurre un comportamento non testato, l'esecuzione viene bloccata.

Marcatura delle righe ineleggibili. Il collatore `CompletionSpanCollator` marca come non eleggibili le righe troncate o in cui lo span della completion non è contiguo. Su quelle righe gli obiettivi ausiliari non vengono applicati, invece di essere applicati a posizioni sbagliate.

Il filo comune è: *fallire in modo rumoroso e presto*, invece di produrre numeri plausibili e sbagliati. È la lezione ricorrente di questo progetto, e il Capitolo 12 ne raccoglie sei casi concreti.

9.4 I criteri di promozione

Entrambi gli obiettivi sono implementati e verificati, ma **non sono stati addestrati**. Non per mancanza di tempo, ma perché sono stati definiti in anticipo dei criteri che devono essere soddisfatti prima di spendere tempo di GPU.

Criterio 1 (massa ammessa). Una sonda in inferenza deve mostrare una riduzione misurabile della *massa rimossa* dal vincolo di decodifica. Cioè: il modello addestrato con l'obiettivo deve collocare più probabilità su token legali, e quindi il Trie deve dover intervenire meno. Questa è la previsione che segue dalla forma funzionale dell'obiettivo, e se non si verifica l'obiettivo non sta facendo ciò che si crede.

Criterio 2 (loss strutturata). Un confronto controllato fra la testa strutturata addestrata con le transizioni *vere* e la stessa testa addestrata con transizioni *mescolate casualmente*. Se le due condizioni danno lo stesso risultato, il guadagno non viene dall'informazione strutturale ma dalla capacità aggiuntiva della testa, e l'obiettivo va abbandonato.

Il secondo criterio è un controllo ablativo classico, e ha una proprietà utile: è *poco costoso* rispetto all'addestramento completo, perché richiede solo la testa e non la messa a punto dell'intero modello.

L'impostazione generale è che un obiettivo ausiliario non va promosso perché è implementato e i test passano. Va promosso quando esiste una previsione quantitativa derivata dalla sua forma funzionale, e quella previsione è verificata su un esperimento di controllo. Su questo progetto, il valore +0,0444 bit della sezione 9.2.3 suggerisce che il margine disponibile sia modesto: è un'informazione che vale la pena avere *prima* di allocare il calcolo, non dopo.

Capitolo 10

Metriche di valutazione

Le metriche non sono uno strumento neutrale di misura: sono parte del disegno sperimentale, e su un corpus saturo come quello descritto nel Capitolo 3 la scelta della metrica determina la conclusione. Questo capitolo documenta la loro implementazione, i loro difetti misurati, e le metriche introdotte per rimediarvi.

L'implementazione è in `src/utils/metrics.py`. Il modulo espone una costante `METRICS_VERSION`, il cui valore corrente è 2: tutti i numeri riportati in questa relazione sono calcolati con la versione 2, e la costante esiste perché confrontare numeri prodotti da versioni diverse del codice di valutazione è un errore facile da commettere e difficile da individuare.

10.1 Il difetto di ROUGE-L, misurato

L'implementazione di ROUGE-L impiegata (che segue la convenzione più diffusa nella letteratura di riferimento, così da restare confrontabile con i numeri pubblicati) esegue due normalizzazioni prima del confronto:

1. porta tutto in minuscolo (*case-insensitive*);
2. spezza i token sui caratteri non alfanumerici.

Su un compito generico sono scelte innocue. Su un compito in cui il gloss è distinto dal testo *proprio* dal maiuscolo e dai prefissi con trattino, sono devastanti. Ecco le misure dirette.

10.1.1 Il maiuscolo non conta

Generato	Riferimento	ROUGE-L
cat sat	CAT SAT	1,00
the cat sat	CAT SAT	0,80

La prima riga è il problema in forma pura: un'uscita in minuscolo, cioè inglese non convertito in gloss, ottiene il punteggio massimo. La seconda mostra che anche mantenendo un articolo che il gloss elimina si resta a 0,80.

Poiché il modello base, non addestrato, produce esattamente inglese in minuscolo, ne segue che ROUGE-L assegna un punteggio alto a un sistema che non ha eseguito *nessuna* conversione. È la spiegazione del risultato apparentemente paradossale della sezione 8.4: il modello senza vincolo ottiene 0,3648 facendo la cosa sbagliata.

10.1.2 I prefissi grammaticali vengono distrutti

La tokenizzazione sui caratteri non alfanumerici spezza i marcatori del gloss:

Stringa	Token prodotti
DESC-GOOD X-WE	['desc', 'good', 'x', 'we']

Il prefisso **DESC-**, che porta informazione grammaticale, diventa un token indipendente. La conseguenza è che due gloss diversi condividono metà dei loro token:

Generato	Riferimento	ROUGE-L
DESC-GOOD	DESC-BAD	0,50
X-WE	X-I	0,50

Confondere *buono* con *cattivo* costa metà del punteggio. Confondere *noi* con *io* costa metà del punteggio. Un sistema che indovina il prefisso e sbaglia sistematicamente la parola ottiene 0,50 su ogni gloss marcato.

10.1.3 La conclusione operativa

ROUGE-L, su questo corpus e con questa implementazione, è una metrica di *copia*: premia la sovrapposizione superficiale di caratteri alfanumerici in minuscolo, che è precisamente la relazione che il corpus sintetico garantisce per costruzione (sezione 3.2).

Non è stata rimossa, per due ragioni: è l'unica metrica su cui esistano numeri pubblicati confrontabili, e la sua degenerazione è essa stessa un risultato da documentare. È stata invece **degradata** da metrica primaria a metrica di confronto storico, e nel codice le metriche di questa famiglia sono raccolte sotto la costante `SATURATED_OVERLAP_METRICS` così che ogni consumatore sappia come trattarle.

10.2 Le metriche primarie

Al posto di ROUGE-L, le metriche promosse a primarie (costante `PRIMARY_METRICS`) sono due.

10.2.1 Exact match normalizzato

Confronto di uguaglianza dopo normalizzazione degli spazi. È binario, severo e non manipolabile: non esiste una strategia che aumenti l'exact match senza produrre l'uscita corretta.

Il suo pregio principale su questo corpus è che *ha ancora risoluzione*. Come mostrato nel Capitolo 5, fra baseline a regole e SFT ROUGE-L distingue 0,0067 mentre l'exact match distingue 0,2265: due ordini di grandezza di differenza nella capacità discriminante.

10.2.2 Non-copy-token accuracy

È la metrica progettata specificamente per questo corpus, e il contributo metodologico principale del capitolo.

L'idea: si considerano soltanto le posizioni in cui il gloss di riferimento *differisce* dal token sorgente maiuscolizzato. Su quelle posizioni, e solo su quelle, si misura l'accuratezza. Formalmente, detto $\mathcal{N} = \{t : r_t \neq \text{upper}(x_t)\}$ l'insieme delle posizioni non banali,

$$\text{non_copy_token_accuracy} = \frac{|\{t \in \mathcal{N} : g_t = r_t\}|}{|\mathcal{N}|}. \quad (10.1)$$

Sui 2000 prompt di valutazione, $|\mathcal{N}| = 9\,350$ token: un campione ampio, non un residuo marginale.

Il potere discriminante è evidente:

Sistema	ROUGE-L	Non-copy accuracy
Base zero-shot senza Trie	0,3648	0,0006
Base zero-shot con Trie	0,1373	0,0432
GRPO da modello base	0,6076	0,4641
Baseline a regole	0,9685	0,9424
SFT	0,9752	0,9660

La colonna di destra ordina i sistemi in modo interpretabile e, soprattutto, assegna al modello base senza vincolo il valore che merita: 0,0006, cioè niente. La colonna di sinistra gli assegna 0,3648, che è più del doppio di quanto ottenga lo stesso modello con il vincolo di decodifica attivo.

10.3 Validity: la definizione, versione 2

La *validity* misura la frazione di uscite ben formate. La definizione è stata rivista dopo aver osservato che la versione iniziale accettava uscite degeneri.

Nella versione 2 un'uscita è dichiarata **non valida** se si verifica almeno una delle condizioni:

1. meno del 50% dei suoi token appartiene al vocabolario gloss;
2. ha più di 4 token e il rapporto fra token distinti e token totali (*unicità*) è inferiore a 0,3.

La prima condizione cattura le uscite che non sono gloss. La seconda cattura le ripetizioni degeneri del tipo **HOUSE HOUSE HOUSE HOUSE HOUSE**, che soddisfano la prima condizione perfettamente (tutti i token sono in vocabolario) e sono comunque prive di valore. La soglia di 4 token evita di penalizzare uscite legittimamente brevi in cui una ripetizione può essere reale.

10.4 Pass@ k

L'implementazione segue lo stimatore combinatorio non distorto (2.18), con soglia di correttezza posta a $\text{ROUGE-L} \geq 0,3$.

La soglia va dichiarata perché il valore di $\text{pass}@k$ dipende interamente da essa e 0,3 è, su questo corpus, una soglia *permissiva*: dato il difetto documentato nella sezione 10.1, un'uscita che supera 0,3 non è necessariamente una traduzione utile. $\text{Pass}@k$ va quindi letta come misura di *copertura* del campionamento, non di qualità.

10.5 La metrica composta e la sua correzione

Una metrica composta comoda è il prodotto fra qualità media e frazione di uscite valide, che penalizza i sistemi che ottengono punteggi alti su poche uscite ben formate:

$$\text{valid_rouge_l_mean} = \overline{\text{rouge}} \times \text{validity_rate}. \quad (10.2)$$

Questa forma ha però un difetto: il prodotto di due medie *non* è la media del prodotto. Se le uscite non valide fossero sistematicamente quelle con ROUGE-L più alto (caso possibile: un'uscita che copia l'inglese può avere ROUGE-L alto e validity nulla), la (10.2) sovrastimerebbe.

È stata quindi affiancata una versione per riga:

$$\text{valid_rouge_l_mean_v3} = \frac{1}{N} \sum_{i=1}^N \begin{cases} \text{rouge}_i & \text{se l'uscita } i \text{ è valida,} \\ 0 & \text{altrimenti.} \end{cases} \quad (10.3)$$

Il confronto fra le due:

Cella	Versione 2	Versione 3
Base few-shot	0,4581	0,4645
Base zero-shot con Trie	0,1243	0,1335

Le differenze sono piccole (+0,006 e +0,009) e nella direzione opposta a quella temuta: la versione per riga dà valori leggermente *più alti*, il che significa che le uscite non valide tendevano ad avere ROUGE-L sotto la media. Entrambe le versioni sono riportate, così che il lettore possa verificare che la scelta fra le due non cambia le conclusioni.

10.6 Incertezza: bootstrap clusterizzato

Tutti gli intervalli di confidenza riportati in questa relazione sono ottenuti per bootstrap, con una precisazione necessaria: il ricampionamento è **clusterizzato per prompt**.

La ragione è che la valutazione genera $N = 5$ uscite per ogni prompt. Le cinque uscite dello stesso prompt non sono osservazioni indipendenti: condividono la frase di ingresso, la sua difficoltà e il suo riferimento. Ricampionare le singole uscite come se fossero indipendenti gonfierebbe la dimensione effettiva del campione di un fattore fino a cinque, restituendo intervalli di confidenza artificialmente stretti.

Il bootstrap clusterizzato ricampiona i *prompt* con reimmissione, portandosi dietro tutte le uscite di ciascun prompt selezionato. È la procedura corretta in presenza di questa struttura a gruppi, e produce intervalli più larghi e onesti. Gli intervalli riportati nella sezione sul confronto fra SFT e baseline a regole (Capitolo 5) sono calcolati in questo modo.

10.7 Il protocollo di valutazione, riassunto

Perché i numeri del capitolo successivo siano interpretabili, il protocollo è fissato e identico per tutte le celle:

Parametro	Valore
Prompt valutati	2000 (dal test set)
Generazioni per prompt	$N = 5$
Strategia di decodifica	campionamento, $\tau = 0,7$
Versione delle metriche	2
Intervalli di confidenza	bootstrap clusterizzato per prompt
Riferimento obbligatorio	baseline a regole

Il campionamento a temperatura 0,7 invece del greedy è una scelta che va motivata: consente di stimare $\text{pass}@k$ e di misurare la varianza delle uscite, che è l'informazione necessaria

per interpretare il supporto dei rollout (sezione 6.4). Il costo è che i numeri di exact match sono leggermente inferiori a quelli che si otterrebbero in greedy; poiché il confronto fra celle usa lo stesso protocollo, i confronti restano validi.

Capitolo 11

Risultati

Tutti i numeri di questo capitolo seguono il protocollo fissato nel Capitolo 10: 2 000 prompt del test set, $N = 5$ generazioni per prompt, campionamento a temperatura 0,7, metriche versione 2.

11.1 La tabella principale

Cella	ROUGE-L	Pass@1	BLEU-s	GlossF1	Validity	EM
Base zero-shot senza Trie	0,3648	—	—	0,2279	0,1043	0,0012
Base zero-shot con Trie	0,1373	0,182	0,030	0,1186	0,9006	0,0017
Base few-shot con Trie	0,4664	0,733	0,188	0,4026	0,9676	0,0148
GRPO da modello base	0,6076	0,9285	0,3205	0,6147	0,9915	0,0528
SFT seguito da GRPO	0,5114	0,796	0,210	0,4702	0,9880	0,0203
SFT soltanto	0,9752	0,9985	0,9436	0,9765	0,9998	0,8130
Baseline a regole	0,9685	—	0,8846	0,9665	—	0,5865
uppercase(text)	0,6454	—	—	—	—	—

La riga in grassetto è il riferimento che rende il resto leggibile. Le osservazioni sostanziali sono cinque, ed è opportuno affrontarle in ordine di importanza decrescente.

11.2 Osservazione 1: nessun risultato con RL supera la regola

Il miglior risultato ottenuto con reinforcement learning è 0,6076 di ROUGE-L (GRPO dal modello base). La baseline a regole ottiene 0,9685. La trasformazione *metti tutto in maiuscolo* ottiene 0,6454.

Cioè: il miglior risultato con RL è **inferiore** a quello ottenibile maiuscolizzando l'input, e distante 0,36 da una tabella di corrispondenze costruita contando parole nel training set.

Su exact match il divario è più netto: 0,0528 contro 0,5865, un fattore undici. Su gloss F1: 0,6147 contro 0,9665.

Non esiste una lettura in cui questi numeri rappresentino un successo del reinforcement learning su questo compito. La domanda residua non è “quanto migliora”, ma “perché non poteva funzionare”; il capitolo sul GRPO ha dato la risposta meccanica, e la sezione 11.7 la quantifica.

11.3 Osservazione 2: l’SFT è al tetto, il che invalida la metrica

L’SFT ottiene ROUGE-L 0,9752, superiore sia alla baseline a regole (0,9685) sia al miglior valore pubblicato in letteratura su ASLG-PC12 (0,9587, [24]).

Un modello da 0,5 miliardi di parametri, con adattatori LoRA, addestrato per tre epoche su una singola GPU, che supera lo stato dell’arte pubblicato. La lettura corretta non è che il nostro sistema sia migliore: è che la metrica ha esaurito la propria capacità discriminante.

Il contrasto con PHOENIX-2014T rende la cosa evidente. Su quel corpus, dove la sovrapposizione lessicale fra gloss e testo è 15,59% invece di 98,23% (sezione 3.2), i migliori risultati T2G riportati sono intorno a ROUGE-L 55,18 [20]. Circa quaranta punti in meno, sullo stesso compito nominale e con metodi comparabili. La differenza non è nei metodi: è nel dato.

11.4 Osservazione 3: il vincolo simbolico e le due metriche

Le prime due righe della tabella principale, lette insieme alla non-copy-token accuracy, sono il risultato neuro-simbolico del lavoro.

Cella	ROUGE-L	Validity	Non-copy accuracy
Base zero-shot senza Trie	0,3648	0,1043	0,0006
Base zero-shot con Trie	0,1373	0,9006	0,0432

Il vincolo abbassa ROUGE-L di 0,23, alza la validity di 0,80 e alza la non-copy accuracy di un fattore 70. Il meccanismo è quello discusso nel Capitolo 8: senza vincolo il modello produce inglese in minuscolo, che la metrica premia per il difetto documentato nella sezione 10.1; con il vincolo produce gloss, male, ma producendo gloss.

La validity di 0,1043 nella prima riga è il dato che chiude la questione: nove uscite su dieci del modello senza vincolo non sono nemmeno sequenze di gloss ben formate. Assegnare loro ROUGE-L 0,3648 è un artefatto della metrica, non una misura di qualità.

11.5 Osservazione 4: le lunghezze non mostrano deriva verso la verbosità

Un’ipotesi corrente sul reinforcement learning è che tenda a produrre uscite più lunghe, perché la reward correla con la lunghezza [32]. Era l’ipotesi di partenza dell’analisi della reward (sezione sul test avversario nel Capitolo 7), e va verificata sui dati.

Cella	Gen.	Rif.	Rapporto	Errore firmato	Errore assoluto
GRPO da base	12,40	12,26	1,011	+0,14	2,81
SFT seguito da GRPO	10,86	12,26	0,886	−1,40	3,59
SFT soltanto	12,32	12,26	1,005	+0,06	0,09
Base zero-shot con Trie	13,96	12,26	1,139	+1,70	—
Base zero-shot senza Trie	10,09	12,26	0,823	−2,17	—

La lettura contraddice l’ipotesi.

- Il GRPO dal modello base produce uscite di lunghezza praticamente corretta in media (rapporto 1,011, errore firmato +0,14). Nessuna deriva verso la verbosità.

- La cella SFT seguito da GRPO produce uscite *più corte* del riferimento (rapporto 0,886, errore firmato $-1,40$). Se ci fosse una pressione della reward sulla lunghezza, agirebbe nella direzione opposta a quella prevista.
- L'errore *assoluto* è la colonna informativa. L'SFT ha 0,09: azzecca la lunghezza quasi sempre. Il GRPO da base ha 2,81 e la pipeline SFT→GRPO ha 3,59, con errori firmati piccoli. Cioè: le celle con RL sbagliano la lunghezza di circa tre token per volta, in entrambe le direzioni, e gli errori si compensano nella media.

La conclusione è che il problema delle celle con RL non è un bias di lunghezza ma una *varianza* di lunghezza. È coerente con l'analisi analitica della componente `edit_validity`, che perde circa 0,13 per token appeso (equazione (7.5)): la reward penalizza attivamente l'allungamento, quindi la verbosità non può essere il canale di guadagno.

Questa è una ritrattazione dichiarata: l'ipotesi "la reward premia la verbosità" era ragionevole a priori, è stata testata due volte (analiticamente e sui dati di lunghezza), e va abbandonata.

11.6 Osservazione 5: la degradazione della pipeline SFT→GRPO

Il risultato più scomodo della tabella:

Cella	ROUGE-L	EM
SFT soltanto	0,9752	0,8130
SFT seguito da GRPO	0,5114	0,0203

Applicare il GRPO a una policy già addestrata la **distrugge**: ROUGE-L da 0,9752 a 0,5114, exact match da 0,8130 a 0,0203. Non un peggioramento marginale, un crollo di quaranta punti di ROUGE-L e di quasi ottanta punti di exact match.

Il meccanismo è quello documentato nella tabella del supporto dei rollout (sezione 6.4). Sulla policy SFT, l'84,8% dei gruppi ha varianza nulla: reward media +0,9462, nessun gruppo al valore minimo, tutti gli otto candidati corretti e con la stessa reward. Su quei gruppi il vantaggio (6.1) è esattamente zero e non c'è aggiornamento utile.

Sul restante 15,2%, invece, c'è varianza, e l'aggiornamento avviene. Ma quel 15% è per costruzione il sottoinsieme in cui la policy SFT era *incerta*, cioè i casi difficili e potenzialmente atipici. Ottimizzare la reward su un campione così selezionato, con la penalità KL come unico freno ($\beta = 0,04$), sposta la policy in una regione che soddisfa meglio la reward e peggiora il riferimento. In assenza di dynamic sampling (sezione sui limiti di TRL nel Capitolo 6) non esiste nessun meccanismo che compensi questa selezione.

Il fenomeno è documentato in letteratura come sovraottimizzazione del modello di reward [4]: la reward è un surrogato imperfetto, e oltre un certo punto migliorarla peggiora l'obiettivo vero. La particolarità qui è che si dispone della misura che lo prevedeva *prima* dell'esperimento: l'84,8% di varianza nulla era osservabile con una singola sonda sui rollout, senza addestrare.

11.7 La decomposizione dell'effetto

La domanda che chiude il capitolo: quanto dell'apparente miglioramento è attribuibile al reinforcement learning?

Si consideri il percorso dal punto di partenza peggiore al miglior risultato con RL:

Passaggio	ROUGE-L	Variazione
Base zero-shot con Trie	0,1373	—
Base few-shot con Trie	0,4664	+ 0,33 (prompting, nessun addestramento)
GRPO da modello base	0,6076	+0,14 (RL, confuso)

Il totale è circa +0,47, di cui:

- **0,33 viene dal prompting**, cioè dall’inserimento di tre esempi recuperati nel prompt. Nessun parametro aggiornato, nessun tempo di GPU per addestramento. Circa il 70% dell’effetto totale.
- **0,14 è attribuito al RL**, ma in modo *confuso*: la cella GRPO da modello base non è isolata dal fattore prompting, e non esiste una cella che vari soltanto il metodo di addestramento tenendo fisso il prompting.

Questa è la confusione di fattori centrale nel disegno originale. Le celle sperimentali variavano simultaneamente metodo di addestramento e modalità di prompting, quindi il contributo dei due non è separabile dai dati raccolti.

La conseguenza per la lettura di questa relazione è precisa: **il fattore sperimentale più efficace misurato in tutto il progetto è il prompting**, e non il metodo di addestramento. Se si dovesse riprogettare l’esperimento, il fattore primario sarebbe la griglia prompting $\times$ decodifica, con il metodo di addestramento come fattore secondario applicato *dopo* aver identificato il substrato con supporto migliore (che, secondo la sezione 6.4, è few-shot con Trie: gruppi morti allo 0,85%).

11.8 Riepilogo per non-copy accuracy

Poiché la non-copy-token accuracy è la metrica con maggiore potere discriminante su questo corpus (Capitolo 10), vale la pena chiudere con l’ordinamento che essa induce:

Sistema	Non-copy accuracy
SFT soltanto	0,9660
Baseline a regole	0,9424
GRPO da modello base	0,4641
Base zero-shot con Trie	0,0432
Base zero-shot senza Trie	0,0006

L’ordinamento è lo stesso che si ottiene con ROUGE-L, con un’eccezione significativa: le prime due righe si stringono (0,9660 contro 0,9424, cioè +0,0236) e le ultime due si separano in modo interpretabile. In particolare, il GRPO da modello base risolve correttamente meno della metà delle posizioni non banali, contro il 94% di una tabella di corrispondenze.

Nessuna delle celle con reinforcement learning si avvicina alla regola. È il risultato del progetto, ed è negativo.

Capitolo 12

Infrastruttura di calcolo

Questo capitolo documenta l'ambiente di esecuzione e, soprattutto, sei guasti reali che ne sono derivati. I guasti sono riportati per esteso perché sono il prodotto più trasferibile del progetto: nessuno di essi era un errore di algoritmo, tutti erano errori di *ordine* o di *ambiente*, e tutti producevano fallimenti silenziosi o diagnostiche fuorvianti.

12.1 La catena di esecuzione

L'addestramento non gira sulla macchina di sviluppo, ma su un cluster gestito da SLURM. Il percorso completo è:

$$\text{login} \rightarrow \text{srun} \rightarrow \text{compute} \rightarrow \text{apptainer} \rightarrow \text{setup} \quad (12.1)$$

login Il nodo di accesso. Non ha il runtime del progetto: nessuna GPU, nessun ambiente Python con le dipendenze. Qualunque comando che importi il codice del progetto fallisce qui.

srun Il comando di allocazione SLURM, che ottiene un nodo di calcolo.

compute Il nodo di calcolo con GPU. Ha il container ma non l'ambiente installato sull'host.

apptainer Il container che fornisce l'ambiente di esecuzione.

setup Lo script che, all'interno del container, prepara l'ambiente.

Il fatto che il nodo di login non abbia il runtime è una fonte ricorrente di confusione, perché il modo naturale di verificare qualcosa ("eseguo un `python -c` per controllare") non funziona lì.

12.1.1 Il container: `exec` e non `run`

Il container viene invocato con `apptainer exec`, non con `apptainer run`. La ragione è concreta: `run` passa attraverso lo *runscript* del container, che manipola gli argomenti, e questa manipolazione corrompe la lista degli argomenti quando si invoca `python -c` con codice che contiene virgolette e spazi. Il sintomo è un errore di sintassi Python su codice sintatticamente valido, che è una diagnostica particolarmente inutile.

L'invocazione è centralizzata nella funzione `run_py` di `cluster/_lib.sh`, che costruisce il comando con dieci direttive `-env` per propagare le variabili d'ambiente necessarie. La centralizzazione è deliberata: se ogni script costruisse la propria invocazione, l'insieme delle variabili propagate divergerebbe fra script e i guasti sarebbero specifici del punto di ingresso.

12.2 Il regime offline

I nodi di calcolo non hanno accesso alla rete. Questo vincolo è più stringente di quanto sembri, perché le librerie moderne di machine learning tentano connessioni di rete per motivi non ovvi: verificare la presenza di aggiornamenti del modello, risolvere un identificativo di dataset, inizializzare la telemetria di un servizio di logging.

L'unico percorso in cui la rete è disponibile è `setup.sh`, che scarica in anticipo modello e dati nella cache locale. Tutti i job successivi girano offline.

La configurazione dell'ambiente offline è raccolta nella funzione `export_offline_env`, che esporta:

Variabile	Effetto
<code>HF_HUB_OFFLINE</code>	disattiva le chiamate all'hub dei modelli
<code>TRANSFORMERS_OFFLINE</code>	idem, a livello di libreria
<code>HF_DATASETS_OFFLINE</code>	idem, per i dataset
<code>WANDB_MODE=offline</code>	registrazione locale, nessun invio
<code>WANDB_DISABLE_WEAVE</code>	disattiva un componente accessorio
<code>WANDB_SILENT</code>	riduce l'output
<code>PYTHONUNBUFFERED</code>	log immediati, essenziale per diagnosticare i job
<code>HF_HOME</code>	radice della cache (<code>\$HOME/.cache/huggingface</code>)
<code>HF_HUB_CACHE</code>	cache specifica dell'hub

Regola critica: `export_offline_env` va invocata *prima di qualunque processo Python*. Le librerie leggono queste variabili al momento dell'import, non all'uso: esportarle dopo il primo import non ha effetto. La sezione 12.4.2 documenta il caso in cui questa regola era violata.

12.3 La catena di job

La quota assegnata consente **una sola allocazione** alla volta. Non è possibile lanciare più esperimenti in parallelo; le celle sperimentali devono essere eseguite in sequenza.

Questo ha richiesto un meccanismo di concatenazione: uno script (`job_chain`) mantiene una coda persistente di celle da eseguire, e a ogni conclusione di job accoda il successivo. Il meccanismo è basato su *tick* periodici, usa `flock` per garantire l'accesso esclusivo alla coda (due tick concorrenti corromperebbero lo stato) e implementa un ritentativo sulle interrogazioni a `sacct`, che sotto carico può non rispondere.

Il monitoraggio è realizzato da `python3 -m src.utils.chain_monitor`, con una modalità `monitor -all` che mostra lo stato di tutta la coda.

12.4 Sei guasti reali

Ciò che segue è la parte utile del capitolo.

12.4.1 Guasto 1: percorso dei dati divergente fra test e produzione

Lo script di preparazione dei dati verificava l'esistenza del percorso `data/aslg_pc12_train`. La produzione, però, usa `data/aslg_pc12`. Il percorso testato non esisteva.

La conseguenza: il controllo di presenza della cache falliva sempre, quindi lo script tentava il *download* del dataset. Su un nodo offline, il download fallisce. Il sintomo era dunque un

errore di rete durante una fase che avrebbe dovuto essere puramente locale, con la diagnostica che indirizzava verso la configurazione offline invece che verso un percorso sbagliato.

Lezione: un controllo di esistenza su un percorso che non è quello usato in produzione è peggio di nessun controllo, perché produce un ramo di fallback che non doveva essere raggiunto.

12.4.2 Guasto 2: ambiente offline esportato dopo il primo Python

Nello script di valutazione, `export_offline_env` era invocata alla riga 177. Il primo processo Python veniva però lanciato alla riga 86, per risolvere la directory di uscita.

Quel processo girava dunque *senza* le variabili offline, tentava una connessione, e in assenza di rete si bloccava fino al timeout. Il job appariva sospeso in una fase di inizializzazione banale.

Lezione: l'ordine delle esportazioni d'ambiente rispetto al primo import è un vincolo di correttezza, non di stile. La contromisura strutturale è invocare la configurazione d'ambiente come primissima istruzione dello script, prima di qualunque logica.

12.4.3 Guasto 3: cambio di firma in Transformers 5.3.0

Questo è il guasto tecnicamente più interessante.

Transformers 5.3.0 ha modificato una funzione interna di rilevamento dei pacchetti, `_is_package_available` che ora restituisce una *tupla* invece di un booleano. Il codice chiamante valutava il risultato in un contesto booleano. Ma in Python

```
bool((False, None)) = True
```

 (12.2)

perché una tupla non vuota è sempre veritiera, indipendentemente dal suo contenuto.

Il risultato: il codice concludeva che il pacchetto di telemetria *era* disponibile, tentava di importarlo, e l'import falliva. Poiché la catena di import passava per `trl.trainer.grpo_trainer`, l'effetto era che **l'intero modulo del trainer GRPO diventava non importabile**, con sette test in rosso e un errore che non menzionava né TRL né il GRPO.

Lezione doppia. Primo: verificare esplicitamente il tipo dei valori restituiti dalle API interne di librerie di terze parti, che non seguono garanzie di compatibilità. Secondo, e più generale: la veridicità implicita in Python trasforma un cambio di tipo in un cambio di comportamento silenzioso. L'esportazione di `WANDB_DISABLE_WEAVE` nella lista della sezione precedente è la contromisura applicata.

12.4.4 Guasto 4: la coda invertita

La funzione di ricostruzione della coda dei job effettuava un *prepend*, cioè inseriva in testa, invece di un *append*.

L'effetto: ricostruendo una coda che doveva essere `[train, eval]` si otteneva `[eval, train]`. La valutazione girava prima dell'addestramento, su un checkpoint inesistente o obsoleto.

Il guasto è insidioso perché entrambi i job *riuscivano*: la valutazione trovava un checkpoint (quello precedente) e produceva metriche plausibili. L'errore si manifestava come risultati sperimentali che non corrispondevano alla configurazione dichiarata, che è la peggiore classe di guasto in un contesto sperimentale.

Lezione: nelle strutture dati che rappresentano un ordine di esecuzione, l'orientamento dell'inserimento va verificato con un test esplicito. Un commento non basta.

12.4.5 Guasto 5: il generatore casuale globale nel curriculum

Già documentato nella sezione 6.5: la classe `CurriculumFilteredDataset` usava il generatore casuale globale del modulo `random`, rendendo la selezione degli esempi dipendente da quante volte il generatore fosse stato consumato altrove nel processo.

La correzione è un generatore dedicato con seed derivato deterministicamente da `(seed, stage)`.

Lezione: in un processo di addestramento, ogni componente che campiona deve possedere il proprio generatore. La condivisione dello stato globale crea accoppiamenti fra parti del codice che non hanno nessuna relazione logica.

12.4.6 Guasto 6: celle di sola valutazione lanciate come addestramento

Le celle `baseline/*` sono celle di *sola valutazione*: misurano il modello base, senza addestrarlo. Le loro configurazioni non contengono quindi la sezione relativa all'addestramento, e in particolare non contengono la chiave `output_dir`.

Lanciandone una con lo script di addestramento, il risultato era:

```
KeyError: 'output_dir'
```

L'errore in sé è corretto: la configurazione non ha quella chiave, perché non deve averla. Il problema era *quando* si manifestava. La validazione avveniva dopo il caricamento di Unsloth e del modello, quindi il job occupava una GPU per minuti prima di fallire con un errore di configurazione diagnosticabile in millisecondi. In un regime a una sola allocazione, ogni minuto sprecato è un esperimento non eseguito.

La correzione è su due livelli, per difesa in profondità:

1. nel punto di ingresso `src/training/__main__.py`, una guardia posta **prima** dell'import di Unsloth. Riconosce le celle di sola valutazione, fallisce in circa un secondo, e il messaggio d'errore indica lo script corretto da usare (`cluster/eval.sh`). L'uscita avviene con codice 2, distinguibile da un fallimento generico;
2. nella funzione principale di `src/training/grpo_t2g_train.py`, la stessa guardia ripetuta, con messaggi distinti per i due casi: cella di sola valutazione contro configurazione genuinamente incompleta.

La copertura è garantita da dieci test in `tests/test_train_entrpoint_guard.py`, che verificano sia l'invariante diretta (una cella di sola valutazione viene rifiutata) sia l'invariante inversa (una cella addestrabile *non* viene rifiutata). Il secondo gruppo è quello che conta di più: una guardia troppo aggressiva rifiuterebbe esperimenti validi, e sarebbe un guasto peggiore di quello che corregge.

Lezione, che è la stessa del guasto 2 in forma generale: **validare prima di allocare**. Ogni controllo che può essere eseguito senza caricare il modello deve essere eseguito prima di caricarlo. È anche il principio codificato nel contratto degli obiettivi ausiliari (sezione 9.3), dove `resolve_auxiliary_config` valida prima del caricamento.

12.5 Il filo comune

I sei guasti hanno cause diverse ma condividono una struttura: nessuno era un errore di matematica, tutti riguardavano l'*ordine* delle operazioni o l'*ambiente* in cui avvenivano.

Guasto	Categoria
1. percorso dati divergente	ambiente: percorso testato $\neq$ usato
2. offline dopo il primo Python	ordine: esportazione dopo l'import
3. tupla veritiera	ambiente: API interna di terze parti
4. coda invertita	ordine: prepend invece di append
5. generatore globale	ambiente: stato condiviso
6. validazione dopo l'allocazione	ordine: controllo dopo il costo

Quattro su sei producevano fallimenti *plausibili*: il job girava, non segnalava nulla di anomalo, e restituiva numeri. Su un progetto sperimentale questa è la categoria di guasto più costosa, perché contamina i risultati senza lasciare traccia nei log.

La contromisura adottata in tutti i casi è la stessa e vale come conclusione del capitolo: **fallire presto e in modo rumoroso**, e codificare ogni invariante scoperta in un test che ne impedisca il ritorno.

Capitolo 13

Organizzazione del codice

Questo capitolo descrive la struttura del codice e i meccanismi che ne garantiscono la coerenza. Serve come mappa per chi voglia riprodurre o estendere il lavoro, e documenta le scelte organizzative che hanno effettivamente prevenuto guasti.

13.1 Struttura dei moduli

Percorso	Responsabilità
<code>src/datasets/</code>	caricamento, suddivisione, retrieval, transizioni
<code>src/models/</code>	caricamento del modello, teste aggiuntive
<code>src/grammar/</code>	Trie, maschera, processore di logits
<code>src/rewards/t2g_rewards.py</code>	le componenti di reward
<code>src/training/</code>	punti di ingresso e trainer (SFT, GRPO, ausiliari)
<code>src/analysis/</code>	baseline a regole, diagnostiche
<code>src/utils/</code>	metriche, configurazione, monitoraggio
<code>cluster/</code>	script SLURM e libreria di shell
<code>remote/</code>	servizio FastAPI e interfaccia testuale
<code>tests/</code>	suite di test (409 test)
<code>experiments/configs/qwen25-05b/</code>	configurazioni delle celle

Due note sulla separazione delle responsabilità.

La prima: `src/grammar/` è indipendente dal modello. Il Trie e la maschera sono strutture dati pure, e tutti i test che ne verificano la correttezza girano senza GPU. Questo ha reso possibile sviluppare e verificare il vincolo di decodifica sulla macchina locale, dove l'addestramento non è eseguibile.

La seconda: `src/analysis/` contiene codice che non partecipa all'addestramento ma è essenziale all'interpretazione. La baseline a regole vive qui, non fra i modelli, perché non è un modello: è uno strumento di misura.

13.2 Il sistema di configurazione

Ogni cella sperimentale è descritta da un file YAML. Il meccanismo di composizione è la chiave `extends`, risolta da `src/utils/config.py`, funzione `resolve_config`, che esegue una *fusione profonda ricorsiva*: i dizionari annidati vengono fusi chiave per chiave, invece di essere sostituiti in blocco.

La distinzione importa. Con una sostituzione in blocco, una cella che volesse cambiare un solo iperparametro dell'ottimizzatore dovrebbe ridichiarare tutta la sezione, e ogni modifica al file di base non si propagherebbe. Con la fusione profonda, una cella dichiara solo le differenze.

Un dettaglio di igiene: la chiave `extends` viene rimossa dalla configurazione risolta e non raggiunge i trainer. Questi ricevono una configurazione già completa e piatta, e non hanno modo di sapere da quale gerarchia provenga. È una separazione voluta: la logica di composizione sta in un solo posto.

13.3 La matrice sperimentale

Il progetto definisce **15 celle**, articolate in **27 voci** di esecuzione (una cella di addestramento genera tipicamente una voce di addestramento e una di valutazione).

Gruppo	Cella	Tipo
baseline	zero-shot	sola valutazione
	zero-shot-no-grammar	sola valutazione
	few-shot	sola valutazione
sft	zero-shot	addestramento
grp	zero-shot	addestramento
	few-shot	addestramento
sft-grp	zero-shot	addestramento
	few-shot	addestramento
ablations/rewards	edit-validity	addestramento
	historical-stack	addestramento
ablations/loss	dr-grp	addestramento
ablations/decoding	no-grammar	addestramento
	hot-rollout	addestramento
ablations/objectives	sft-allowed-mass	addestramento
	sft-structured	addestramento

Le tre celle **baseline** sono di sola valutazione: misurano il modello non addestrato. Le loro configurazioni non contengono la sezione di addestramento, ed è precisamente questa asimmetria che ha prodotto il guasto documentato nella sezione 12.4.6.

Le cinque celle di ablazione sono le più interessanti dal punto di vista metodologico, perché ciascuna isola un fattore che la parte principale dell'esperimento confonde:

- **edit-validity** contro **historical-stack** isola l'effetto della revisione dello stack di reward (Capitolo 7);
- **dr-grp** attiva esplicitamente la variante di denominatore che nessuna configurazione storica selezionava (sezione 6.3);
- **no-grammar** isola il contributo del vincolo simbolico (Capitolo 8);
- **hot-rollout** varia la temperatura di generazione dei candidati, cioè il parametro che governa direttamente il supporto dei rollout (sezione 6.4);
- le due celle in **objectives** corrispondono ai due obiettivi ausiliari del Capitolo 9.

13.4 La validazione delle configurazioni

Il file `tests/validate_configs.py` è il punto in cui le invarianti scoperte durante il progetto sono codificate. Non è un test di stile: ogni controllo corrisponde a un guasto reale o a un attacco dimostrato.

Controllo	Motivazione
presenza delle sezioni attese	un errore di struttura deve fallire subito
tipi dei valori	impedisce coercizioni silenziose
somma dei pesi di reward = 1,0	la scala della reward non deve variare fra celle
chiavi morte rifiutate	impedisce configurazioni che sembrano attive e non lo sono
knob dell'obiettivo RL validi	rende esplicita la variante ottimizzata
peso del termine fuori vocabolario in $[0; 0,6]$	sopra 0,6 esiste un attacco dimostrato
<code>max_prompt_length</code> ≥ 512 con retrieval	un prompt troncato annulla il few-shot

Vale la pena soffermarsi su due voci.

13.4.1 Le chiavi morte

Durante la pulizia del progetto sono state rimosse tre componenti di reward (sezione 7.4) e diversi percorsi legacy. Il rischio, dopo una rimozione, è che una configurazione continui a impostare una chiave che non esiste più: il file appare configurato in un certo modo, il codice ignora la chiave, e l'esperimento gira con impostazioni diverse da quelle dichiarate.

Il validatore rifiuta esplicitamente le chiavi rimosse. È una forma di protezione contro la classe di guasto più costosa in un contesto sperimentale: la discrepanza fra configurazione dichiarata e comportamento effettivo.

13.4.2 Il vincolo sulla lunghezza del prompt

Il controllo `max_prompt_length` ≥ 512 in presenza di retrieval merita una nota, perché protegge da un fallimento perfettamente silenzioso. Se il prompt viene troncato, gli esempi recuperati vengono eliminati, e la cella few-shot diventa indistinguibile da una cella zero-shot. Nessun errore, nessun avviso: solo numeri che sembrano dire che il few-shot non serve.

Dato che il fattore prompting è, secondo la sezione 11.7, l'effetto più grande misurato in tutto il progetto, un guasto che lo annullasse silenziosamente sarebbe particolarmente grave.

13.5 Le diagnostiche di Markov

Il modulo `src/analysis/markov_diagnostics.py` implementa strumenti di ispezione sui percorsi di transizione fra gloss:

Funzione	Ruolo
<code>path_log_energy</code>	punteggio logaritmico di un percorso
<code>hard_viterbi_diagnostic</code>	percorso di massima verosimiglianza
<code>soft_viterbi_diagnostic</code>	versione smussata (somma-log-esponenziale)

Due scelte implementative vanno segnalate.

La prima: le funzioni sono validate contro un calcolo a forza bruta su istanze piccole. Un algoritmo di programmazione dinamica come Viterbi è facile da implementare in modo

sottilmente sbagliato (un indice fuori posto produce risultati plausibili), e il confronto con l'enumerazione esaustiva su casi di dimensione ridotta è la verifica che coglie quel tipo di errore.

La seconda: esiste una guardia che rifiuta spazi di stato superiori a 256 elementi. È un limite deliberato, non un difetto: queste funzioni servono a ispezionare casi specifici, e permetterne l'uso su spazi grandi le trasformerebbe in un collo di bottiglia computazionale mascherato da strumento di analisi.

Il punto più importante è lo *statuto* di questo modulo. Le quantità di Viterbi sono state **rimosse dalle reward** (sezione 7.4) perché a lunghezza uguale collassano su un segnale già presente. Sono invece state **conservate come diagnostica**. La distinzione è sostanziale: una quantità può essere informativa da ispezionare e inutile, o addirittura dannosa, da ottimizzare.

13.6 Il servizio remoto

La directory **remote/** contiene un servizio FastAPI e un'interfaccia testuale per interagire con il cluster: sottomettere celle, ispezionare la coda, recuperare risultati.

La sua ragione d'essere è il vincolo descritto nel Capitolo 12: la macchina di sviluppo non può eseguire l'addestramento, e il nodo di login non ha il runtime del progetto. Ogni operazione sperimentale passa dunque per una connessione remota, e avere un'interfaccia unica riduce la superficie di errore rispetto a comandi costruiti a mano.

13.7 La suite di test

Lo stato corrente è di 409 test. La loro distribuzione riflette la strategia adottata: la maggior parte verifica componenti eseguibili senza GPU.

Strutture simboliche. Il Trie, la maschera, il calcolo dell'insieme ammesso. Nessuna dipendenza dal modello.

Reward. Ogni componente valutata su candidati costruiti a mano, compresi quelli avversari del Capitolo 7.

Metriche. Compresi i casi che documentano i difetti di ROUGE-L (sezione 10.1): quei valori sono asseriti nei test, in modo che il difetto sia registrato e non possa essere “corretto” per distrazione rendendo i numeri non confrontabili.

Contratti degli obiettivi ausiliari. L'inerzia esatta a peso nullo e i rifiuti espliciti delle configurazioni incompatibili (sezione 9.3).

Guardie dei punti di ingresso. I dieci test della sezione 12.4.6, con l'invariante diretta e quella inversa.

13.7.1 Un test rosso dichiarato

Un test risulta fallito nella suite corrente: `tests/test_remote_app.py::test_ssh_alias_no_user_no_port`.

Il fallimento è stato verificato presente anche sul codice non modificato: non è una regressione introdotta dal lavoro descritto in questa relazione. La causa è che il test effettua una connessione SSH reale verso il cluster, quindi il suo esito dipende dalla disponibilità della rete e dalla configurazione dell'ambiente di esecuzione, non dal codice.

È riportato qui, invece di essere silenzioso, perché lo stato della suite fa parte dei risultati: dichiarare “409 test, tutti verdi” quando uno è rosso sarebbe una piccola falsificazione, e un

test che dipende dalla rete è un difetto di disegno del test da correggere separatamente, non da nascondere.

13.8 La pulizia del codice

Una parte sostanziale del lavoro è consistita nella rimozione di codice. Le voci principali:

Elemento rimosso	Note
Automa a stack e grammatica LL(1)	mai eseguito, sovradimensionato (sez. 8.3)
Tre componenti di reward basate su Viterbi	ridondanti (sez. 7.4)
Infrastruttura di supporto	circa 8 774 righe complessive
Percorsi di checkpoint legacy	puntavano a directory non più prodotte
Configurazioni obsolete	sostituite dalla matrice corrente
Sei documenti di progetto	superati o contraddittori

La rimozione di codice funzionante richiede giustificazione, e in ciascun caso è stata fornita: l'automa a stack non era mai stato eseguito e aveva un conflitto di parsing irrisolto; le reward basate su Viterbi erano dimostrabilmente ridondanti sia algebricamente sia empiricamente (entro $\pm 0,006$ dal controllo); i percorsi legacy puntavano a directory che nessuna cella corrente produce.

Il criterio applicato è che il codice non eseguito è un costo senza contropartita: va mantenuto solo se esiste un piano concreto per eseguirlo. Gli obiettivi ausiliari del Capitolo 9 sono l'eccezione deliberata, e per questo sono accompagnati da criteri di promozione espliciti.

Capitolo 14

Conclusioni

14.1 Che cosa è stato stabilito

Il lavoro non produce un miglioramento delle metriche. Produce cinque affermazioni sostenute da misure, che vale la pena enunciare nella loro forma più compatta.

14.1.1 Le metriche su ASLG-PC12 sono sature

Non nel senso vago di “i punteggi sono alti”, ma nel senso preciso che una baseline a regole di poche righe, che non apprende nulla, ottiene ROUGE-L 0,9697 e supera ogni risultato ottenuto con reinforcement learning (Capitolo 3). Persino la trasformazione `uppercase(text)` ottiene 0,6454, cioè batte il miglior risultato con RL (0,6076).

La causa è documentata: il corpus è sintetico (sezione 3.2), la sovrapposizione fra token del gloss e del testo è 98,23% contro il 15,59% di un corpus annotato da umani [24], e conoscere il token sorgente rimuove circa il 90% dell’incertezza sul gloss (0,856 bit residui su 8,578).

La metrica stessa contribuisce: l’implementazione di ROUGE-L impiegata è insensibile al maiuscolo e spezza i token sui caratteri non alfanumerici, quindi assegna 1,00 a un’uscita in minuscolo identica al testo di partenza (sezione 10.1).

Il segno più chiaro della saturazione è che un modello da 0,5 miliardi di parametri, con adattatori LoRA, addestrato per tre epoche su una singola GPU, supera il miglior valore pubblicato su questo corpus.

14.1.2 La domanda di ricerca iniziale era mal posta

La domanda “il GRPO migliora la generazione di gloss?” non è rispondibile su ASLG-PC12, perché presuppone che esista un margine di miglioramento su cui il reinforcement learning possa agire. Con l’SFT a exact match 0,8130 e validity 0,9998, e una baseline a regole a ROUGE-L 0,9685, quel margine non esiste.

La domanda si riduce, su questo corpus, a “il GRPO memorizza una mappa quasi-identità meglio dell’SFT?”, che è una questione di ottimizzazione e non riguarda la lingua dei segni.

14.1.3 Il risultato negativo ha un meccanismo in tre parti

Questo è il contributo sostanziale. Non “il GRPO non ha funzionato”, ma tre misure che spiegano *perché* non poteva funzionare.

Parte 1: il collasso del supporto. Il gradiente del GRPO è proporzionale al vantaggio (6.1), che si annulla quando tutti i candidati di un gruppo ricevono la stessa reward. Sul modello base senza vincolo di decodifica il 73,9% dei gruppi è morto; sulla policy SFT

l'84,8% dei gruppi ha varianza nulla per la ragione opposta, cioè perché tutti i candidati sono corretti (sezione 6.4). Il GRPO è schiacciato fra due fallimenti simmetrici, e la finestra utile su un compito quasi deterministico è stretta.

Parte 2: la degradazione della pipeline. Applicare il GRPO alla policy SFT porta ROUGE-L da 0,9752 a 0,5114 ed exact match da 0,8130 a 0,0203. Il meccanismo è che gli aggiornamenti avvengono soltanto sul 15,2% dei gruppi con varianza, che è per costruzione il sottoinsieme in cui la policy era incerta; ottimizzare la reward su un campione così selezionato, con la sola penalità KL come freno, sposta la policy fuori dalla regione corretta. È sovraottimizzazione del surrogato [4], prevedibile con una singola sonda sui rollout prima di addestrare.

Parte 3: la confusione dei fattori. Dei circa 0,47 di ROUGE-L che separano il punto di partenza peggiore dal miglior risultato con RL, 0,33 vengono dal *prompting* (few-shot con retrieval, nessun parametro aggiornato) e circa 0,14 sono attribuiti al RL in modo non isolato (sezione 11.7). Il fattore sperimentale più efficace misurato in tutto il progetto non è il metodo di addestramento.

Va aggiunto un chiarimento sui limiti di questa affermazione. TRL 0,24,0 non implementa il *dynamic sampling* di DAPO [40], che scarta i gruppi a varianza nulla e ricampiona, né il *clip-higher*; e la variante di denominatore effettivamente ottimizzata (**dapo**) è stata selezionata per omissione e non per scelta (sezione 6.3). Quindi nessuna delle mitigazioni note contro il collasso del supporto era attiva. Il risultato negativo riguarda questa configurazione, non il GRPO in generale.

14.1.4 Il vincolo simbolico abilita l'apprendimento senza migliorare le metriche

Il risultato neuro-simbolico, e il più controintuitivo. Il Trie (Capitolo 8) fa *crollare* ROUGE-L da 0,3648 a 0,1373 e contemporaneamente:

- alza la non-copy-token accuracy da 0,0006 a 0,0432, un fattore circa 70;
- alza la validity da 0,1043 a 0,9006;
- porta i gruppi morti dal 73,9% al 19,9%, cioè *crea* il supporto su cui il reinforcement learning può operare.

L'interpretazione: il vincolo restringe la distribuzione generativa alla regione in cui la reward è discriminante. Fuori da quella regione i candidati sono tutti ugualmente cattivi e il vantaggio li annulla tutti.

La conseguenza pratica ha valore generale. In un esperimento di reinforcement learning su uscite strutturate, il vincolo simbolico non è un accessorio di qualità: è una preconditione di fattibilità, e va valutato misurando il supporto dei rollout invece delle metriche finali. Il costo che il vincolo impone sulla qualità del contenuto è reale e documentato in letteratura [28, 42, 37]; la conclusione di questo lavoro è che su un compito con supporto scarso quel costo va pagato, perché l'alternativa è un gradiente nullo.

14.1.5 L'onestà metodologica come parte del risultato

Tre ritrattazioni misurate sono documentate in questa relazione, e sono parte del contributo.

1. L'ipotesi "la reward premia la verbosità", ragionevole a priori e documentata in letteratura [32], è stata **refutata** due volte: analiticamente (l'equazione (7.5) è decrescente, con perdita misurata di circa $-0,13$ per token appeso) ed empiricamente (i rapporti di lunghezza delle celle con RL sono 1,011 e 0,886, cioè non c'è deriva). Il problema delle celle con RL è la varianza di lunghezza, non il bias.
2. La stima del prior strutturale è stata **corretta due volte** prima di arrivare a $+0,0444$ bit, per contaminazione fra insiemi e per un backoff mal specificato (sezione 9.2.3). Un guadagno così piccolo può essere prodotto interamente da un errore di metodo, e chi legge deve sapere che l'errore è stato cercato.
3. Le tre componenti di reward basate su Viterbi sono state **rimosse** dopo aver dimostrato che a lunghezza uguale collassano algebricamente sullo stesso segnale, e che empiricamente restano entro $\pm 0,006$ dal valore di controllo (sezione 7.4).

Va dichiarato anche un test rosso: `test_ssh_alias_no_user_no_port` fallisce, per una dipendenza dalla rete nel test stesso, e il fallimento è stato verificato presente anche sul codice non modificato (Capitolo 13).

14.2 Lavoro futuro

Le direzioni seguenti derivano direttamente dai risultati, in ordine di priorità.

14.2.1 Cambiare corpus

La prima e più importante. Il compito T2G va valutato su PHOENIX-2014T, dove la sovrapposizione lessicale fra gloss e testo è 15,59% invece di 98,23% e i migliori risultati riportati sono intorno a ROUGE-L 55,18 [20]. Lì esiste un margine reale, e le domande sul reinforcement learning diventano rispondibili.

Il *caveat* va dichiarato in anticipo: PHOENIX-2014T è tedesco e riguarda la lingua dei segni tedesca. Il cambio di corpus comporta un cambio di lingua sorgente, quindi la scelta del modello base e l'analisi degli artefatti vanno rifatte da zero. Non è un trasferimento gratuito.

Una direzione più radicale, coerente con la critica al gloss come rappresentazione [39, 38], è di abbandonare l'intermedio testuale. Esistono lavori che applicano il rinforzo alla traduzione della lingua dei segni senza passare per il gloss [27], usando un segnale di qualità definito su rappresentazioni visive. In quel contesto il problema del supporto dei rollout andrebbe riesaminato da zero, perché lo spazio delle uscite e la reward sono di natura completamente diversa; ma la domanda sarebbe posta sul compito reale invece che su una sua trascrizione impoverita.

14.2.2 Ridisegnare l'esperimento primario

L'esperimento primario dovrebbe essere una griglia

$$\text{prompting} \times \text{decodifica} \tag{14.1}$$

con il metodo di addestramento come fattore *secondario*. La ragione è diretta: sono i due fattori con l'effetto misurato più grande ($+0,33$ il prompting, e il vincolo che porta i gruppi morti dal 74% al 20%), e sono gli unici due che il disegno attuale confonde fra loro.

Il metodo di addestramento va applicato *dopo* aver identificato il substrato con il supporto migliore, che secondo la sezione 6.4 è few-shot con Trie (gruppi morti allo 0,85%, reward media $-0,3233$).

14.2.3 Eseguire i gate degli obiettivi ausiliari

I due obiettivi del Capitolo 9 sono implementati, verificati e non addestrati. I criteri di promozione sono definiti:

1. per la massa ammessa, una sonda in inferenza che mostri una riduzione misurabile della massa rimossa dal vincolo di decodifica;
2. per la loss strutturata, un confronto controllato fra testa addestrata con transizioni vere e con transizioni mescolate casualmente.

Entrambi i gate costano poco rispetto a un addestramento completo, ed entrambi possono produrre un risultato negativo che risparmia l'esperimento. Il valore $+0,0444$ bit della qualificazione strutturale suggerisce che il margine disponibile sia modesto.

14.2.4 Il reinforcement learning, se e quando

L'ordine corretto delle operazioni, ricavato per via negativa da questo lavoro, è: prima si stabilisce un substrato con supporto misurato, poi si progetta la reward con test avversari, poi si addestra. Non prima.

La verifica di fattibilità è a costo quasi nullo: si generano G candidati su un campione di prompt, si calcolano le reward, e si contano i gruppi a varianza nulla. Se la frazione è alta, il reinforcement learning non apprenderà, indipendentemente dagli iperparametri. È una misura che richiede minuti e che avrebbe risparmiato, in questo progetto, una quantità considerevole di tempo di GPU.

14.3 La lezione trasferibile

Se il lavoro ha un unico insegnamento riutilizzabile, è il seguente.

Prima di ottimizzare una metrica, misurare quanto la si può ottenere senza apprendimento; e prima di addestrare con reinforcement learning, misurare se il segnale esiste.

Le due misure hanno costo trascurabile. La prima è una tabella di corrispondenze contata sul training set, che qui ha rivelato ROUGE-L $0,9697$ e ha reso interpretabile ogni numero successivo. La seconda è una sonda sui rollout, che qui ha rivelato il $73,9\%$ di gruppi morti prima dell'addestramento e l' $84,8\%$ di varianza nulla dopo l'SFT, e che avrebbe previsto entrambi i risultati negativi.

Nessuna delle due era presente nel disegno originale. Entrambe, in retrospettiva, erano le prime cose da fare.

Bibliografia

- [1] Dario Amodei, Chris Olah, Jacob Steinhardt, Paul Christiano, John Schulman, and Dan Mané. Concrete problems in AI safety. *arXiv preprint arXiv:1606.06565*, 2016.
- [2] Timothee Cour, Ben Sapp, and Ben Taskar. Learning from partial labels. *Journal of Machine Learning Research*, 12:1501–1536, 2011.
- [3] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. QLoRA: Efficient finetuning of quantized LLMs. *arXiv preprint arXiv:2305.14314*, 2023.
- [4] Leo Gao, John Schulman, and Jacob Hilton. Scaling laws for reward model overoptimization. *arXiv preprint arXiv:2210.10760*, 2022.
- [5] Saibo Geng, Martin Josifoski, Maxime Peyrard, and Robert West. Grammar-constrained decoding for structured NLP tasks without finetuning. *arXiv preprint arXiv:2305.13971*, 2023.
- [6] Kartik Goyal, Chris Dyer, and Taylor Berg-Kirkpatrick. An empirical investigation of global and local normalization for recurrent neural sequence models. *arXiv preprint arXiv:1904.06834*, 2019.
- [7] Daya Guo et al. DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.
- [8] Kelvin Guu, Panupong Pasupat, Evan Zheran Liu, and Percy Liang. From language to programs: Bridging reinforcement learning and maximum marginal likelihood. *arXiv preprint arXiv:1704.07926*, 2017.
- [9] Chris Hokamp and Qun Liu. Lexically constrained decoding for sequence generation using grid beam search. *arXiv preprint arXiv:1704.07138*, 2017.
- [10] Ari Holtzman, Jan Buys, Li Du, Maxwell Forbes, and Yejin Choi. The curious case of neural text degeneration. *arXiv preprint arXiv:1904.09751*, 2019.
- [11] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685*, 2021.
- [12] Huang et al. On the effectiveness of globally normalized models for sequence labeling. *arXiv preprint arXiv:2106.03376*, 2021.
- [13] Kimi Team. Kimi k1.5: Scaling reinforcement learning with LLMs. *arXiv preprint arXiv:2501.12599*, 2025.
- [14] Chin-Yew Lin. ROUGE: A package for automatic evaluation of summaries. In *Text Summarization Branches Out*, 2004.

- [15] Zichen Liu et al. Understanding R1-zero-like training: A critical perspective. *arXiv preprint arXiv:2503.20783*, 2025.
- [16] Amit Moryossef et al. An open-source gloss-based baseline for spoken to signed language translation. *arXiv preprint arXiv:2305.17714*, 2023.
- [17] Amit Moryossef, Kayo Yin, Graham Neubig, and Yoav Goldberg. Data augmentation for sign language gloss translation. *arXiv preprint arXiv:2105.07476*, 2021.
- [18] Andrew Y. Ng, Daishi Harada, and Stuart Russell. Policy invariance under reward transformations: Theory and application to reward shaping. In *Proceedings of the 16th International Conference on Machine Learning (ICML)*, 1999.
- [19] Achraf Othman and Mohamed Jemni. English-ASL gloss parallel corpus 2012: ASLG-PC12. In *Proceedings of the 5th Workshop on the Representation and Processing of Sign Languages, LREC*, 2012.
- [20] Younes Ouargani and Noussaima El Khattabi. Advancing text-to-GLOSS neural translation using a novel hyper-parameter optimization technique. *arXiv preprint arXiv:2309.02162*, 2023.
- [21] Long Ouyang et al. Training language models to follow instructions with human feedback. *arXiv preprint arXiv:2203.02155*, 2022.
- [22] Kishore Papineni, Salim Roukos, Todd Ward, and Wei-Jing Zhu. BLEU: A method for automatic evaluation of machine translation. In *Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2002.
- [23] Kanghee Park, Jiayu Wang, Taylor Berg-Kirkpatrick, Nadia Polikarpova, and Loris D’Antoni. Grammar-aligned decoding. *arXiv preprint arXiv:2405.21047*, 2024.
- [24] Yuqi Peng, Jian Zeng, and Hai Zhao. Better sign language translation with monolingual data. *arXiv preprint arXiv:2304.10844*, 2023.
- [25] Maja Popović. chrF: Character n-gram f-score for automatic MT evaluation. In *Proceedings of the Tenth Workshop on Statistical Machine Translation*, 2015.
- [26] Matt Post and David Vilar. Fast lexically constrained decoding with dynamic beam allocation for neural machine translation. *arXiv preprint arXiv:1804.06609*, 2018.
- [27] Rao et al. RVLF: A reinforcing vision-language framework for gloss-free sign language translation. *arXiv preprint arXiv:2512.07273*, 2025.
- [28] Reddy, Walker, Ide, and Bedi. The hidden cost of structured generation in LLMs: Draft-conditioned constrained decoding. *arXiv preprint arXiv:2603.03305*, 2026.
- [29] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.
- [30] Junghoon Seo and Joon Suk Huh. On the power of deep but naive partial label learning. *arXiv preprint arXiv:2010.11600*, 2020.
- [31] Zhihong Shao et al. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.
- [32] Prasann Singhal, Tanya Goyal, Jiacheng Xu, and Greg Durrett. A long way to go: Investigating length correlations in RLHF. *arXiv preprint arXiv:2310.03716*, 2023.

- [33] Harry Walsh, Ben Saunders, and Richard Bowden. Select and reorder: A novel approach for neural sign language production. *arXiv preprint arXiv:2404.11532*, 2024.
- [34] Sean Welleck, Ilia Kulikov, Stephen Roller, Emily Dinan, Kyunghyun Cho, and Jason Weston. Neural text generation with unlikelihood training. *arXiv preprint arXiv:1908.04319*, 2019.
- [35] Ronald J. Williams. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine Learning*, 8:229–256, 1992.
- [36] Yadav et al. Limits of difficulty scaling: Hard samples yield diminishing returns in GRPO-tuned SLMs. *arXiv preprint arXiv:2604.06298*, 2026.
- [37] Ye et al. Efficient and asymptotically unbiased constrained decoding for large language models. *arXiv preprint arXiv:2504.09135*, 2025.
- [38] Kayo Yin, Amit Moryossef, Julie Hochgesang, Yoav Goldberg, and Malihe Alikhani. Including signed languages in natural language processing. *arXiv preprint arXiv:2105.05222*, 2021.
- [39] Kayo Yin and Jesse Read. Better sign language translation with STMC-transformer. *arXiv preprint arXiv:2004.00588*, 2020.
- [40] Qiying Yu et al. DAPO: An open-source LLM reinforcement learning system at scale. *arXiv preprint arXiv:2503.14476*, 2025.
- [41] Urchade Zaratiana, Nadi Tomeh, Pierre Holat, and Thierry Charnois. Filtered semi-markov CRF. *arXiv preprint arXiv:2311.18028*, 2023.
- [42] Zhou. From hallucination to structure snowballing: The alignment tax of constrained decoding in LLM reflection. *arXiv preprint arXiv:2604.06066*, 2026.